Multimodal Knowledge Base — operating manual and visual demonstration

English edition • 4 November 2026 • Local repository snapshot

This guide follows the reference interface manual, with additional completed workflows, recorded browser demonstrations, API coverage and reproducible inputs. On-screen labels remain Chinese so that English readers can find the controls.

Evidence scope: The refreshed screenshots and interface tour were captured from this repository at `http://127.0.0.1:5176`, with Flask on `5004` and the populated Neo4j graph on Bolt port `7688`. Graph browsing and search were read-only. The import and library recordings use the same repository with Flask on `5002`. The monitoring capture uses an isolated InfluxDB instance on `8087`. Automatic vector rebuilding was disabled (`DISABLE_KB_BOOTSTRAP=1`). The saved index summary records 138 documents and 5,719 chunks; visible file counts may differ after demonstration uploads.

Confidence is **high** for the captured graph, fresh Q&A, diagnosis and monitoring workflows. All demonstration measurements are synthetic.

Contents

1. Quick start and workflow map
2. Home, navigation and appearance
3. Import and upload history
4. PDF parsing and multimodal processing
5. Completed spreadsheet import and editing
6. Dataset management, previews and recovery
7. Vector retrieval and index maintenance
8. Knowledge graph
9. Evidence-based questions
10. Diagnosis: three input modes
11. Saved reports and source evidence
12. Condition monitoring
13. Accounts and management
14. Recordings and screen grabs
15. API examples and endpoint catalogue
16. Command-line tools and troubleshooting

1. Quick start and workflow map

Use an existing project environment (`venv/` or `.venv/`). Never install project dependencies in conda base. In separate terminals:

```
source venv/bin/activate
python app.py
```

```
cd frontend
npm run dev
```

Default development frontend: `http://127.0.0.1:5173`. Backend port comes from `.env`; the supplied example uses `5001`. Vite proxies `/api` and `/pdfs` to that backend. The refreshed tour used isolated capture ports `5176/5004`. A built `frontend/dist` can also be served by Flask.

Main workflow: import one file → extraction/cleaning → compare source and Markdown → edit if needed → **保存并入库** → find the document in **数据集管理** → use retrieval/graph/Q&A → enter an anomaly in **诊断报告** → open its saved report.

Destination	Route	Main action
首页	`/`	Asset overview, shortcuts and recent questions
知识导入	`/parse`	Upload history and new import
Import workspace	`/parse/create`	Upload, inspect, edit, save
数据集管理	`/dataset`	Filter, preview, rename and delete
知识图谱	`/graph`	Search, inspect relations and evidence chains
知识问答	`/qa`	Ask questions and inspect citations
诊断报告	`/diagnosis`	Run anomaly diagnosis
Saved report	`/reports/<reportId>`	Reopen a persistent report
状态监测	`/timeseries`	Import and visualize equipment series
平台管理	`/manage`	Current management entry screen

`/vector`, `/vector/documents` and `/vector/documents/:docKey` redirect to `/dataset`. `VectorPage.vue` exists in the source tree but is not registered as a reachable page.

2. Home, navigation and appearance

Open **首页**. The page presents the knowledge workflow, asset composition, processing-state counts, category distribution, condition-monitoring area and suggested/recent questions. Scroll down for module cards; **进入系统** and module links lead into workspaces. Summary, quality and distribution are supplied by distinct APIs, so their counts can differ.

Use the top navigation to change work areas. The theme switch toggles light/dark appearance and stores the choice in browser local storage. **设置 → 用户管理** opens `/manage`; **退出** currently returns to the home page.



Home after data loading



Light appearance



Settings menu

3. Import and upload history

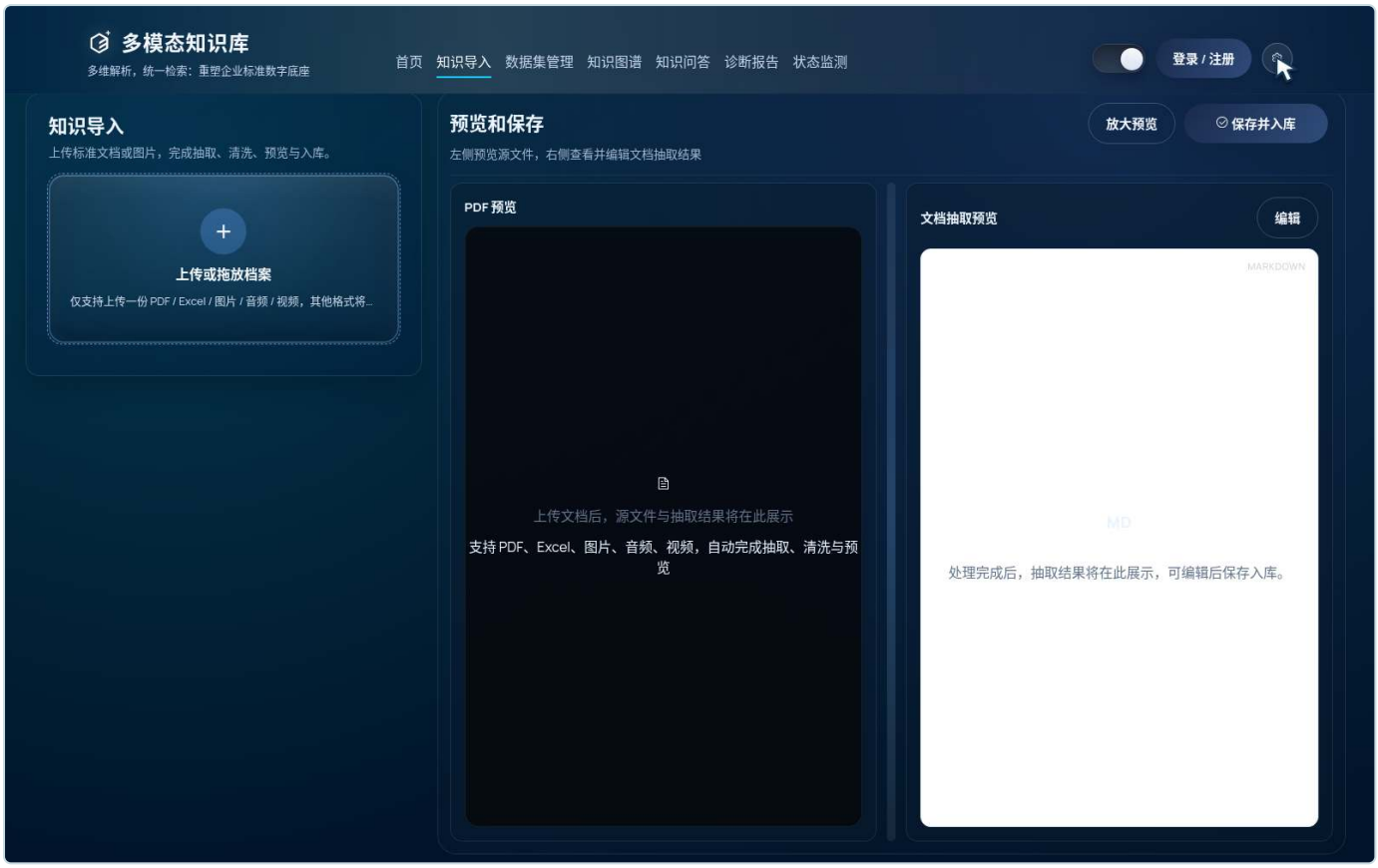
Open **知识导入**. **开始导入** opens the upload workspace. The history table shows file name/type, destination category, upload time, state and available location actions. Use its search field, type filter (PDF/image/Excel/audio/video) and state filter (completed/processing/failed) to narrow the list. **查看定位** opens the corresponding dataset file when one has been committed.

In `/parse/create`, click or drag a single supported file into the upload zone. Fill **文档名称**, select **保存栏目**, or select **+ 新增栏目** and enter a name. Remove an incorrectly queued file before submitting. PDF inputs additionally expose **解析策略**. Start processing using the upload action, then follow the timeline and log entries.

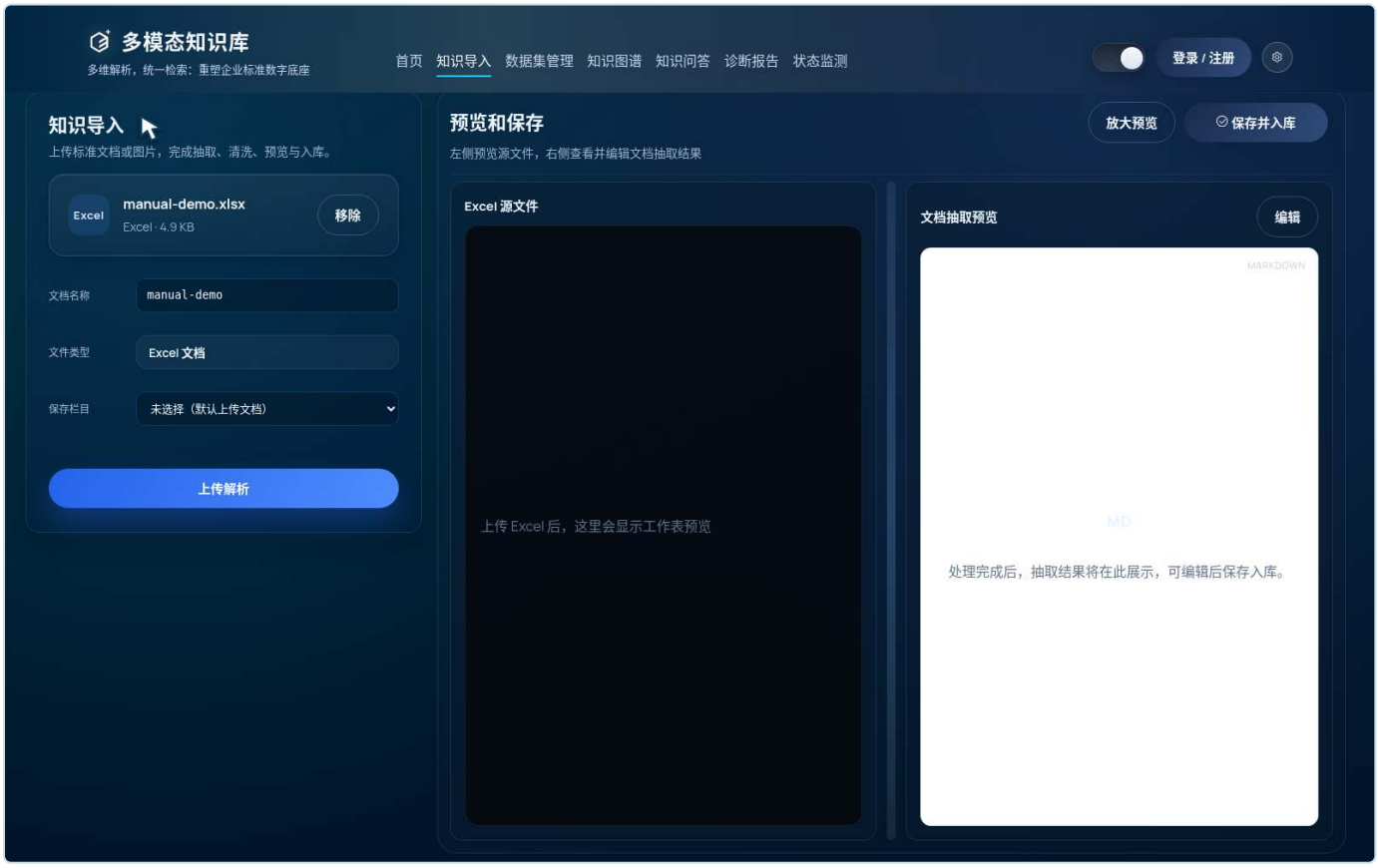
The file picker accepts PDF, Excel (`xlsx/xls/xlsm`), common images, audio and video extensions. Each displayed upload is one file. Internal queue code does not imply a multi-file batch picker.



Import history and workflow



New import workspace



Selected file and destination controls

4. PDF parsing and multimodal processing

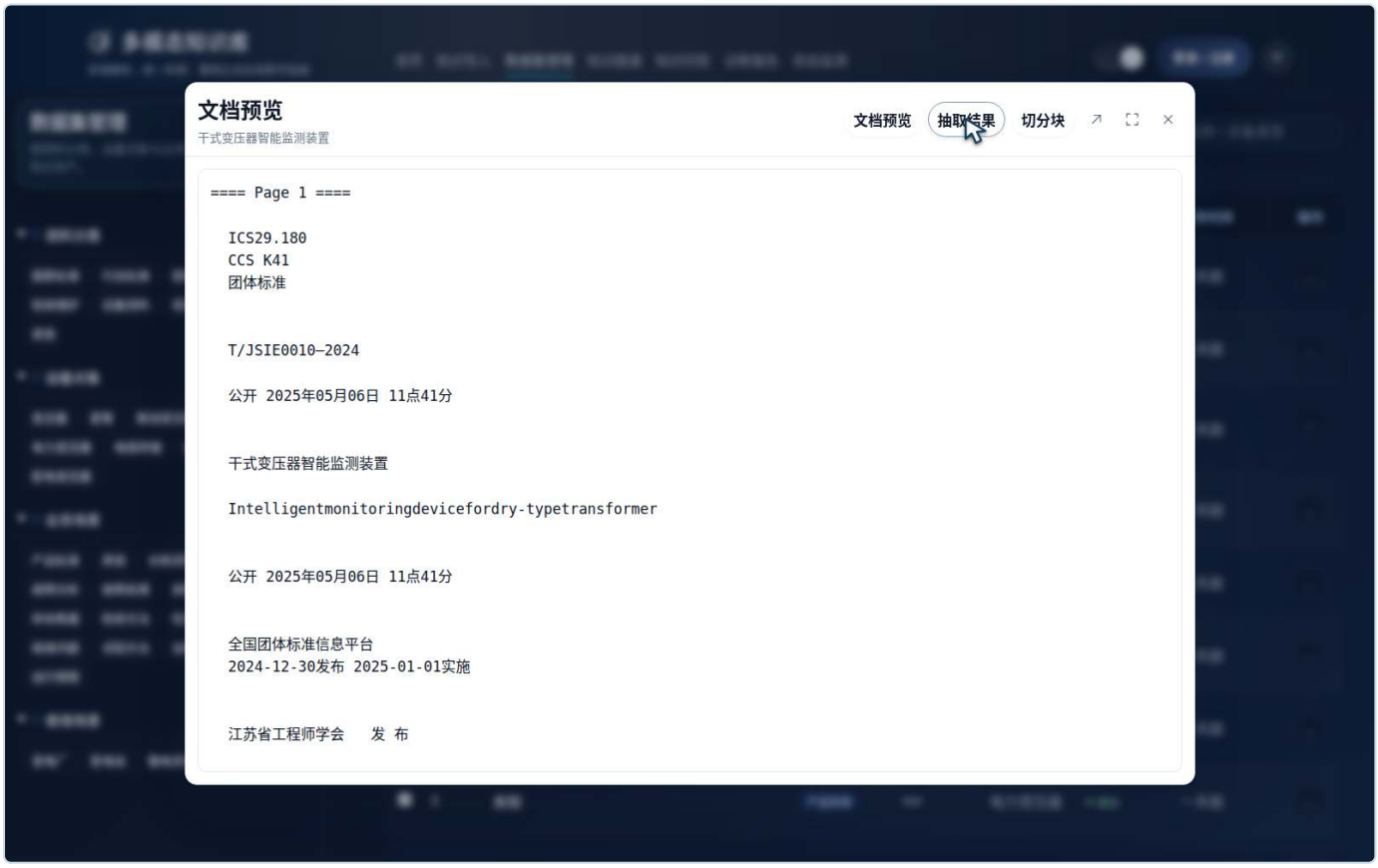
Input	Processing and operator action	Verification in this session
PDF	**自动（推荐）**, **高保真文本（PyPDF）**, **复杂版面（Docling）**, or **扫描件全页 OCR**. Compare source and extracted text; change strategy and use **重新解析** for an eligible uncommitted task.	Existing source PDF, extracted text and chunks opened; new OCR/Docling runs not performed.
Excel	Parse workbook sheets into Markdown tables; inspect the sheet preview and extracted text.	New two-row workbook processed, edited and committed.
Image	Local Ollama or configured remote vision processing produces Markdown; compare with source image.	Existing source image preview opened; new model inference not run.
Audio	Speech recognition produces a transcript, then an LLM organizes cleaned Markdown.	Source-verified implementation; no audio sample present for execution.
Video, remote	Default remote path extracts audio and returns its ASR transcript.	Source-verified implementation; not executed.
Video, local	`MM_VIDEO_PROVIDER=local` or `ollama`: FFmpeg samples frames; local vision model describes frames and produces Markdown.	Source-verified implementation; not executed.

Remote video processing is **transcript-only** in the current implementation; it does not also perform visual defect detection. Local video processing uses frame summaries. FFmpeg/ffprobe and the selected model service are required for these paths.

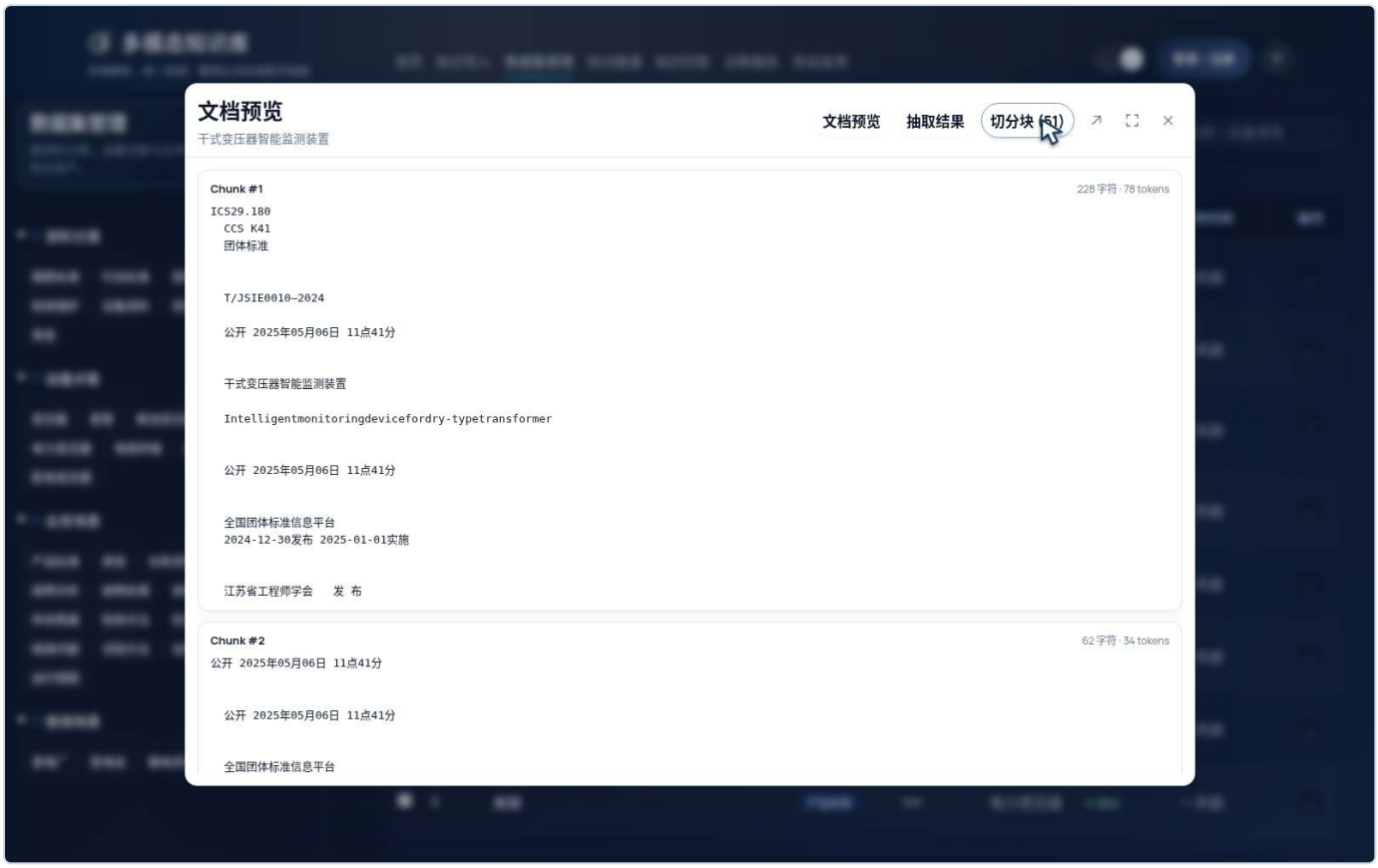
The preview supports source PDFs, images, workbook sheets and native audio/video players when a source exists. **编辑 → 保存** updates the extraction draft; **保存并入库** is a separate step that commits the source and cleaned/edited Markdown. Saving triggers incremental knowledge ingestion, whose vector/graph success depends on its services.



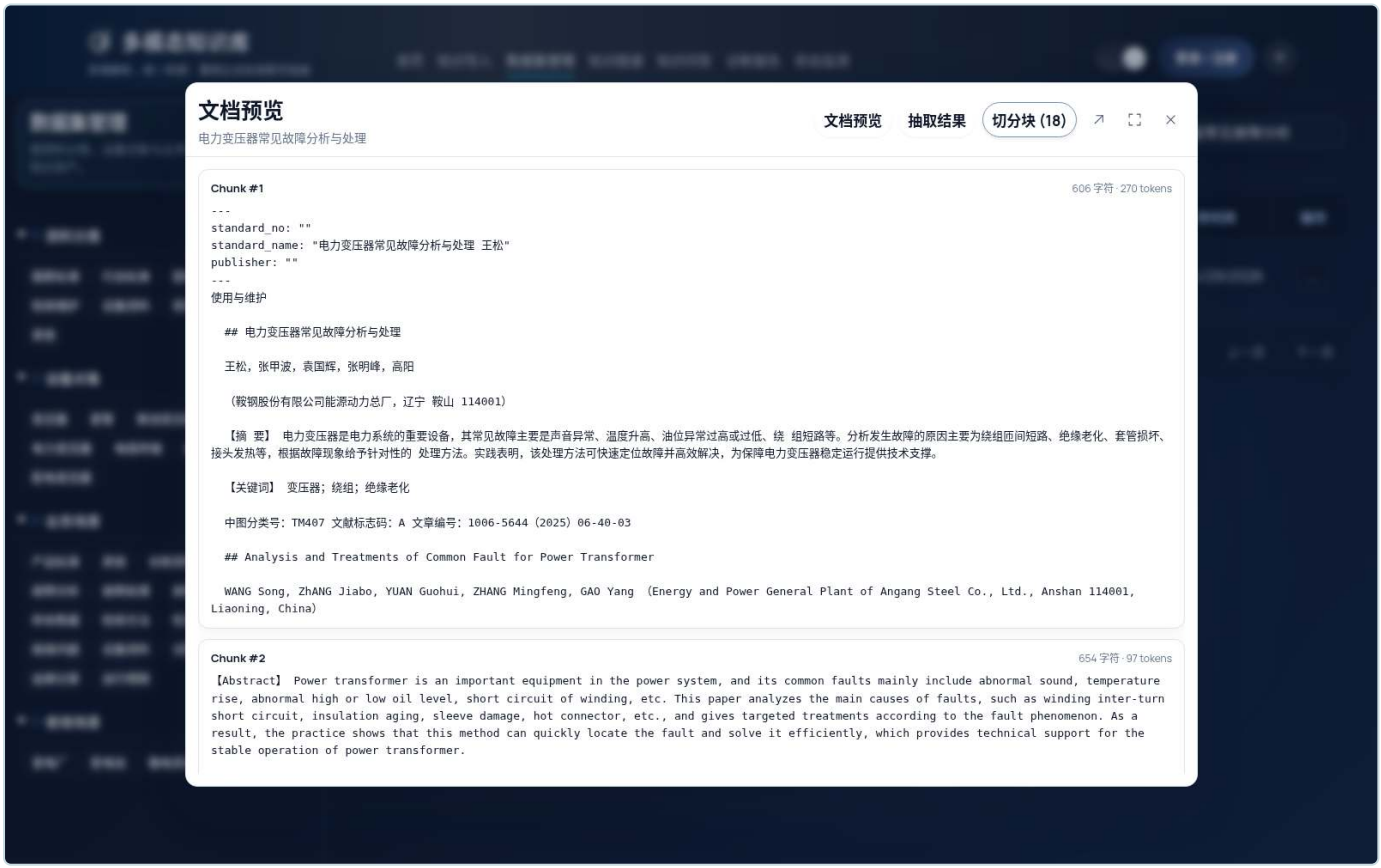
Existing PDF source preview



Existing extracted Markdown



Empty chunk state for one existing document



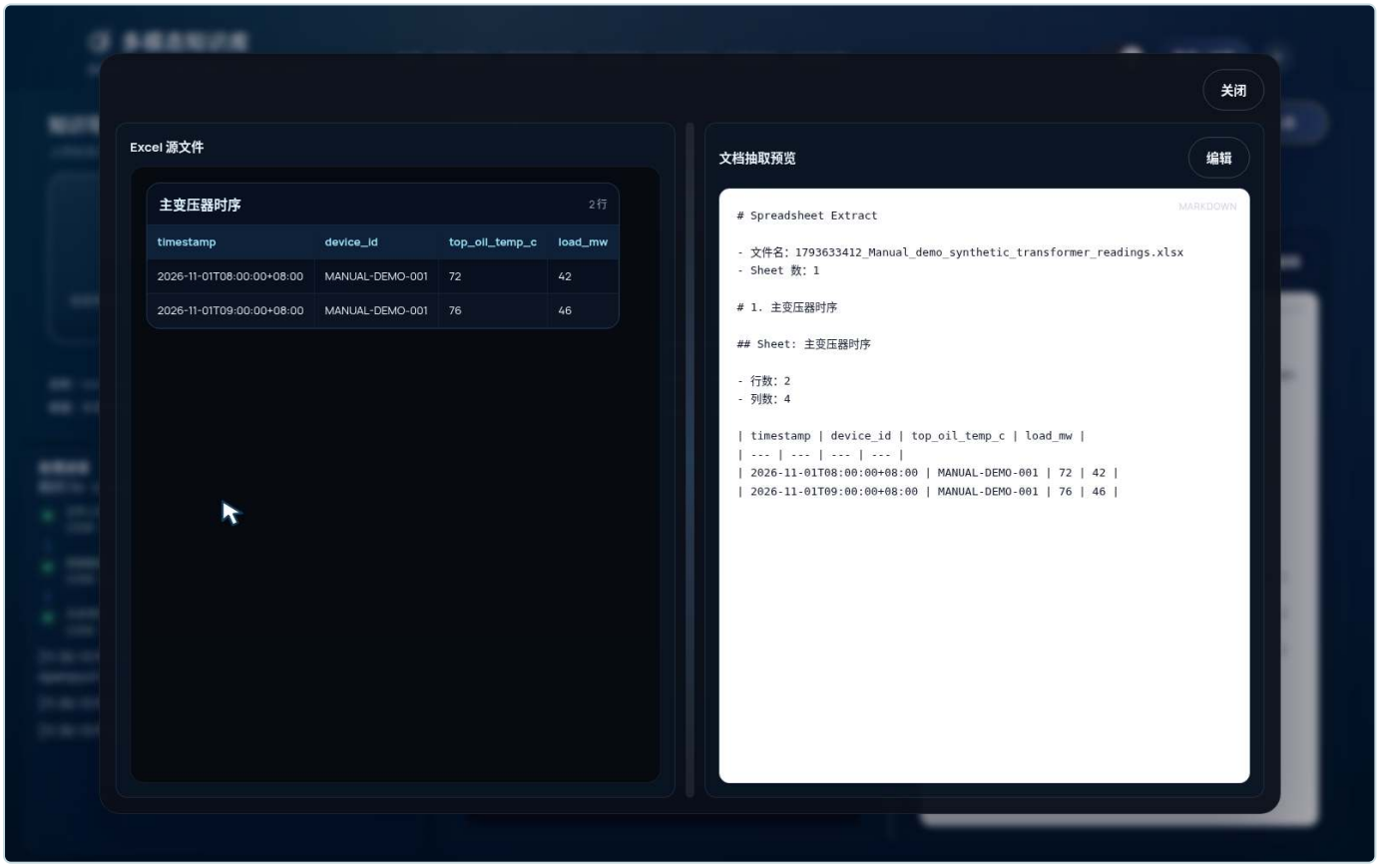
Populated chunk list from another existing document

5. Completed spreadsheet import and editing

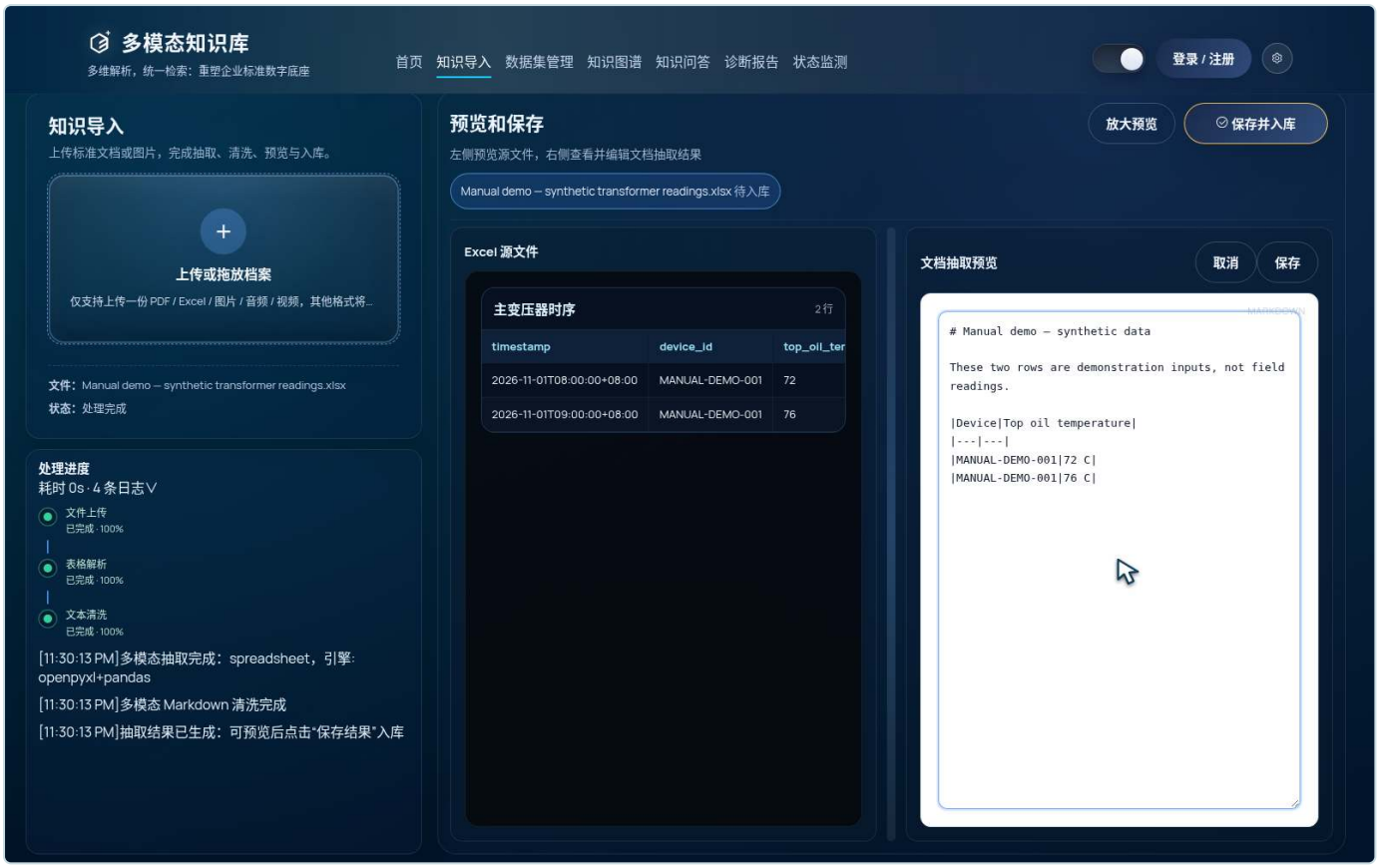
Reproduce with manual-demo.xlsx. The sheet **主变压器时序** contains two synthetic readings for `MANUAL-DEMO-001`: 72/76°C and 42/46 MW, at 08:00/09:00 on 1 November 2026 (+08:00).

1. Upload the workbook and name it `Manual demo — synthetic transformer readings`.
2. Wait for **表格解析** and **文本清洗** to complete. The observed engine was `openpyxl+pandas`.
3. Review the source table beside the Markdown table. Processing may open an enlarged preview automatically; use **关闭** to return to the workspace.
4. Click **编辑**, amend the Markdown, then **保存**. The demonstrated edit explicitly labels the readings synthetic.
5. Click **保存并入库**. Find the saved workbook in the dataset, reopen it, and verify **抽取结果** contains the edited draft.

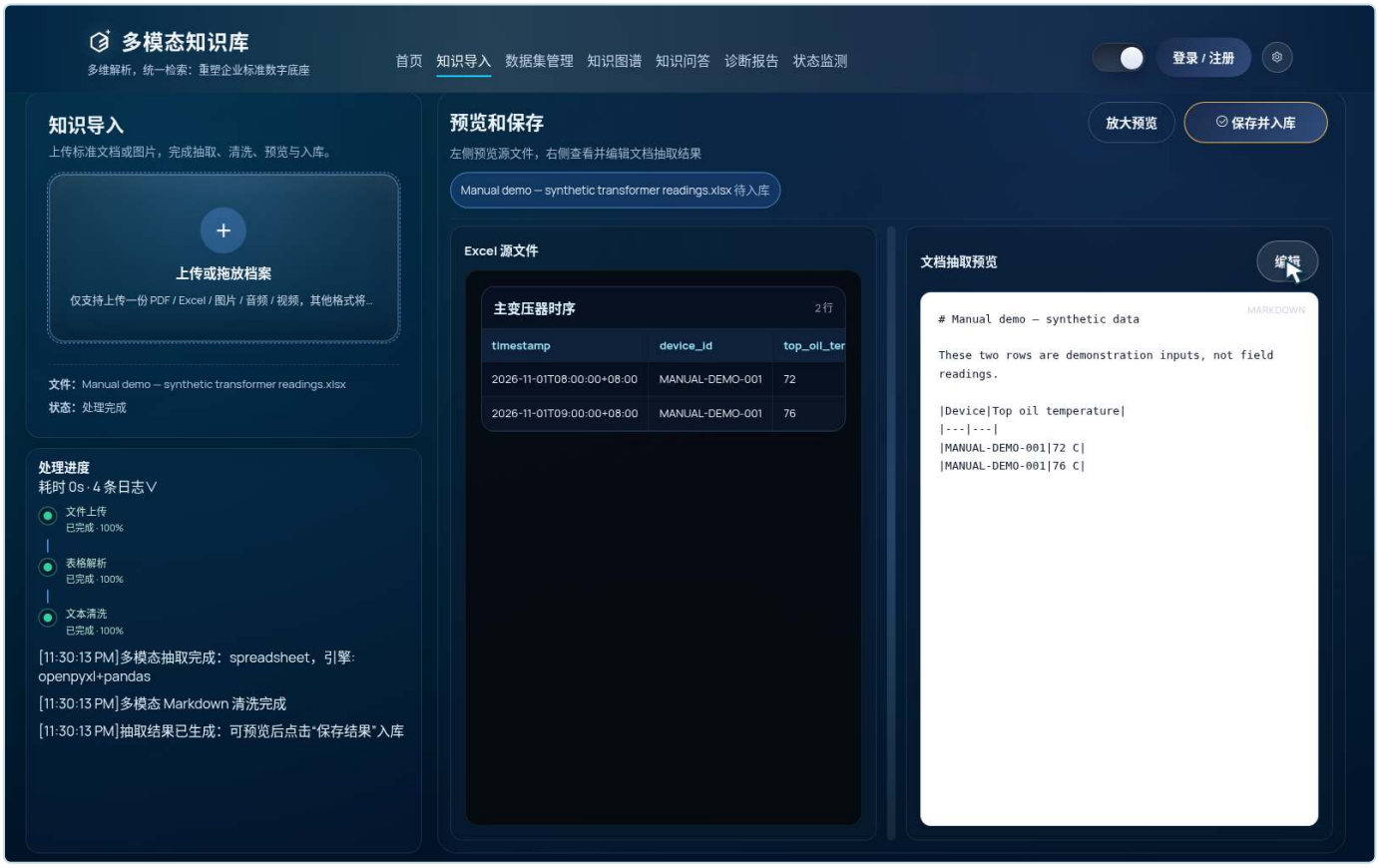
Observed result: the source workbook remained readable and the edited Markdown was retained after commit. For local indexing of new documents, set `EMBEDDING_DOCUMENT_MODEL=voyageai/voyage-4-nano`. A separate isolated append test added one document to a copy of the 5,719-vector Voyage 4 index, producing 5,720 vectors and ranking the new document first for its test query. The recorded workbook upload was not rerun for this indexing check. Knowledge-library import is separate from **状态监测** import into InfluxDB.



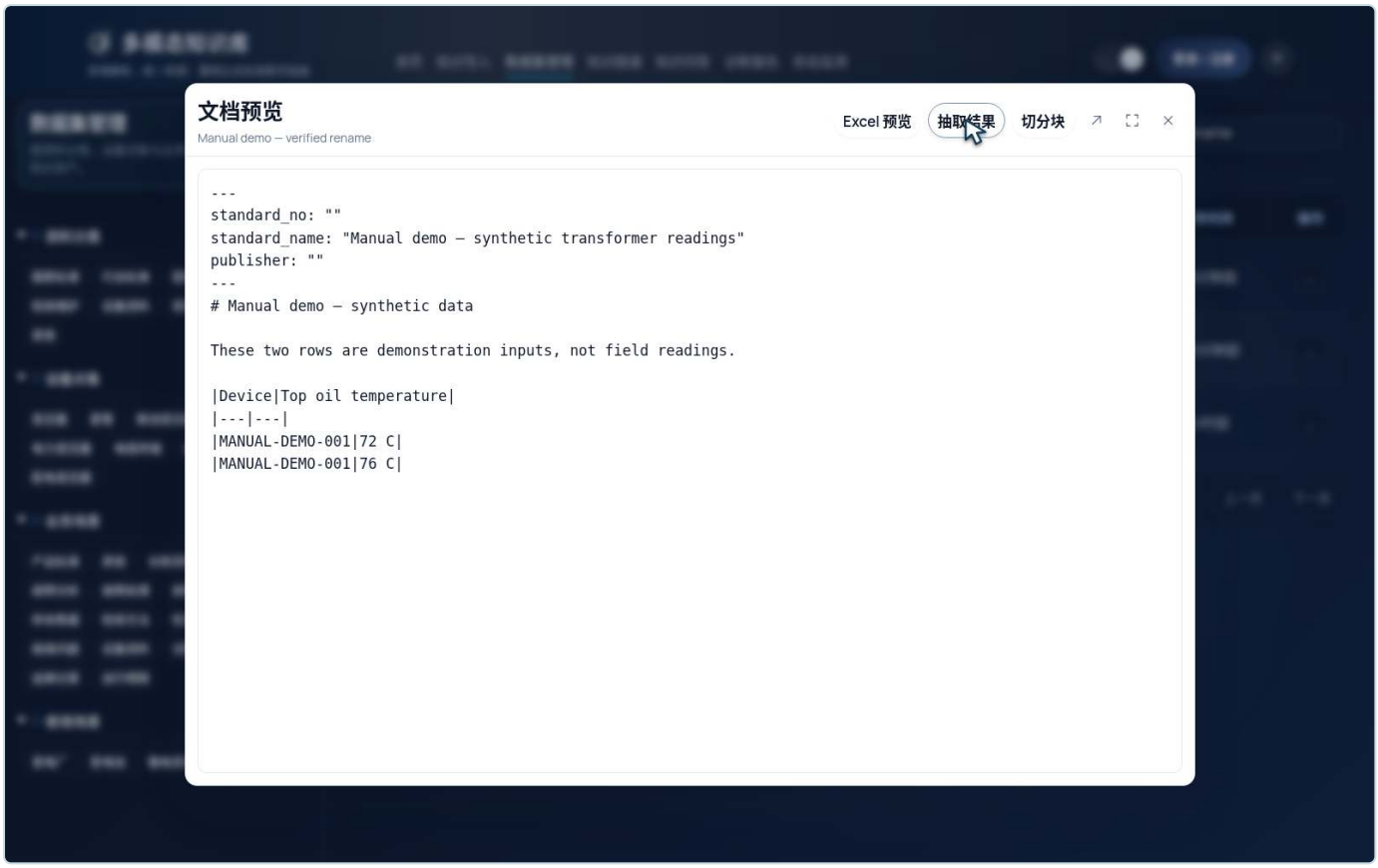
Actual workbook extraction



Editing the extraction draft



Saved draft before commit



Edited content reopened from the library

6. Dataset management, previews and recovery

Open **数据集管理**. Filter using **资料分类 → 设备对象 → 业务场景**, with **使用场景** as an additional facet. Search by document name/device type. Switch between **文档** and **图片**, collapse the sidebar, use **上一页/下一页**, or select rows with checkboxes. Selection alone does not expose a batch operation in this template.

Click a document name to open its preview. Use source preview, **抽取结果**, **切分块**, open-source-file, fullscreen and close controls. PDFs use an iframe; images show the original; workbooks show sheet tabs and cell tables. Some existing entries display **未知** because their title metadata is incomplete; use source/category context to identify them.

Open **… → 编辑文档名称**, change the title, then **保存**; **取消** or Escape abandons the edit. The title-map change does not rename the underlying source file. **… → 删除** soft-deletes the source and associated Markdown to the local trash and excludes it from visible results.

Demonstrated: the synthetic workbook was renamed `Manual demo — verified rename`, soft-deleted, restored using the actual backend recovery endpoint, and reopened. No pre-existing document was deleted. Recovery uses `POST /api/pdfs/restore` with `{"file":"<relative source path>"}`. A recovery drawer exists in the component, but the current template has no visible button opening it; use the API until that entry is wired. Trash records store a 30-day `purge_after_ts`; that field is not proof that a scheduled purge job runs.



Document library



Search for the synthetic workbook



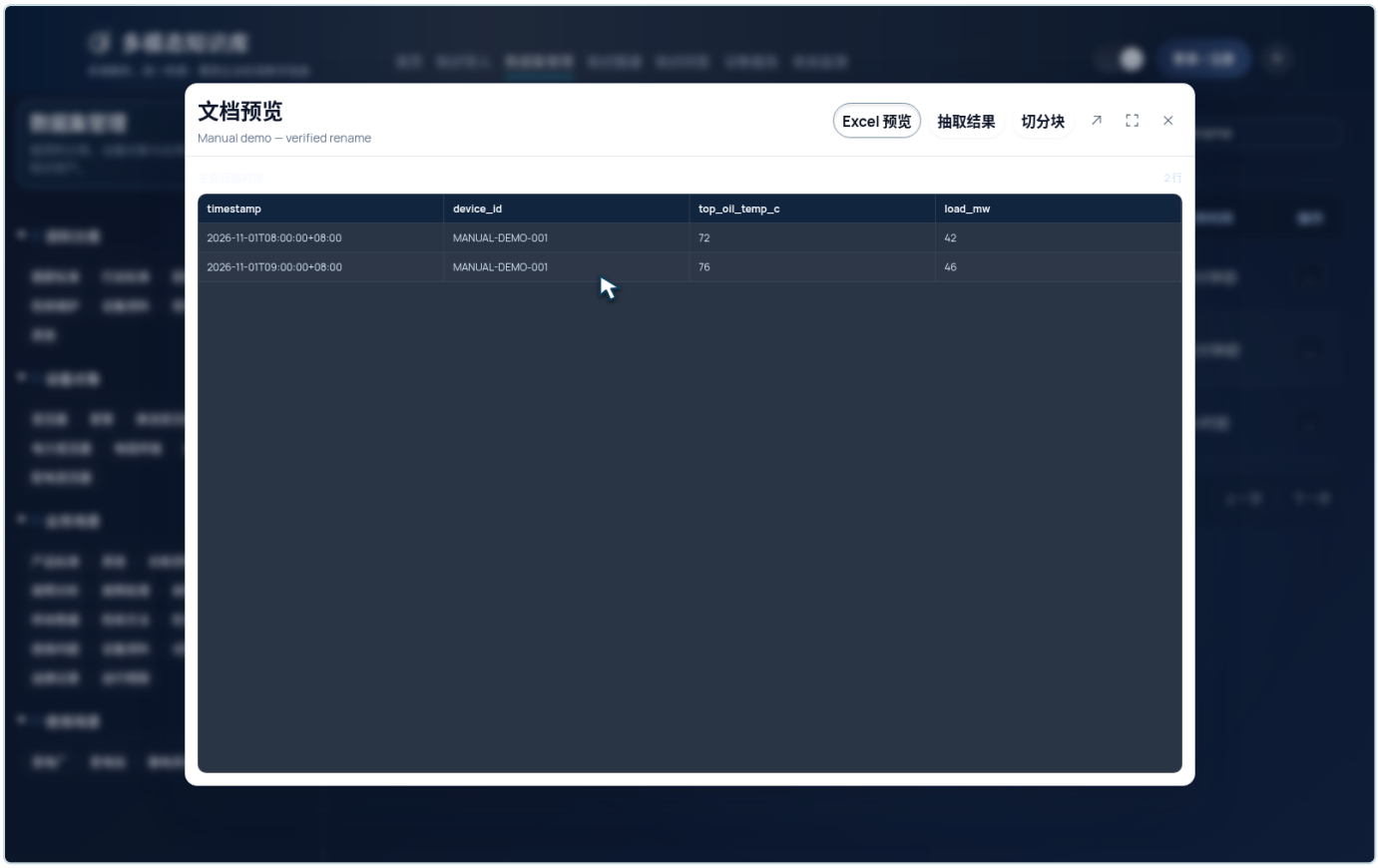
Rename editing



After deleting only the demo file



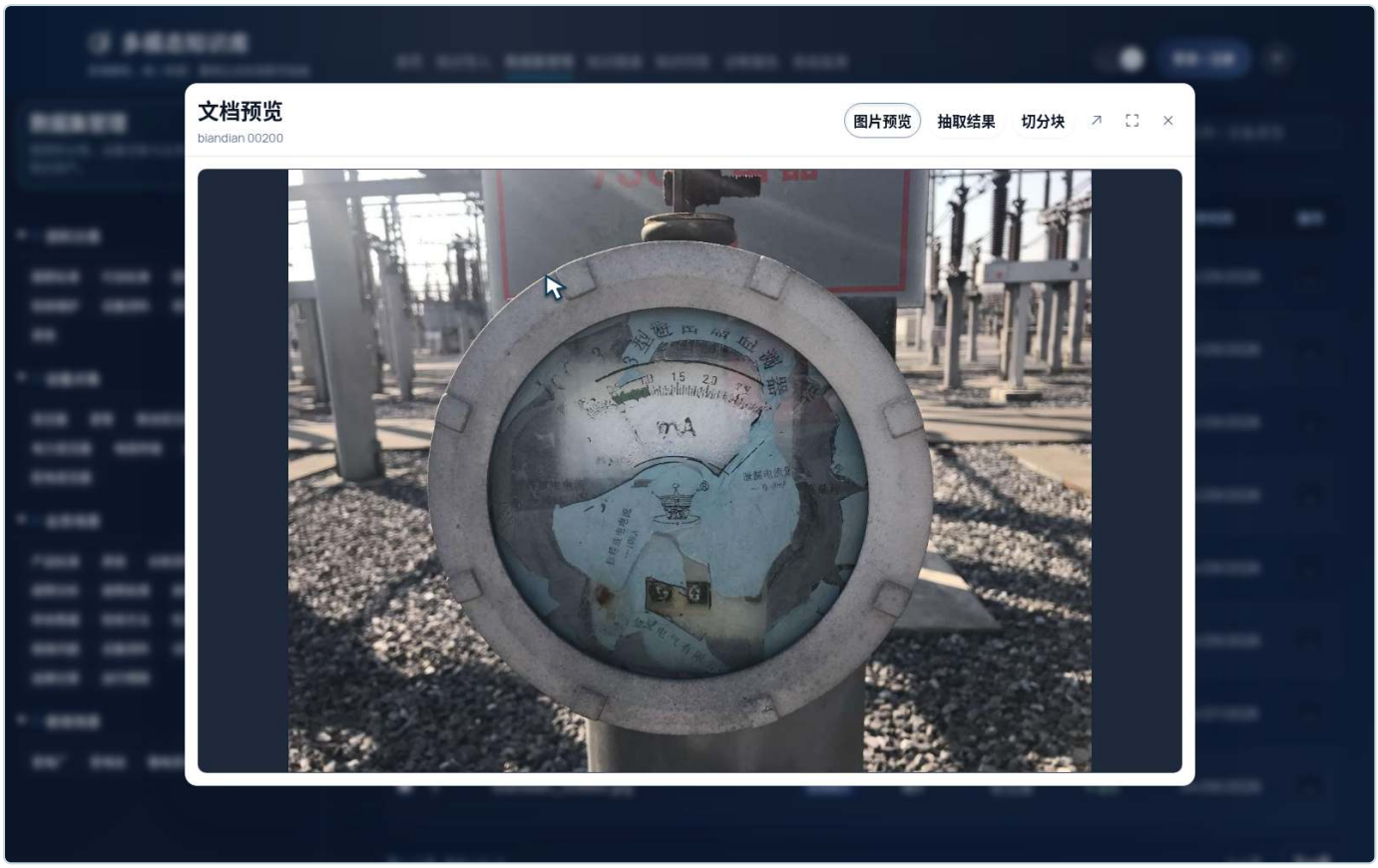
Demo file restored



Workbook source preview



Image library



Original image preview



Category filter



Pagination



Collapsed sidebar

7. Vector retrieval and index maintenance

There is no separate vector workbench in the active router. Inspect chunks in the dataset preview, or use the backend endpoints:

- `GET /api/vector/overview`: index summary, distributions and sample chunks.
- `GET /api/vector/docs`: document-level indexing information.
- `GET /api/vector/doc-chunks?doc_id=<id>&limit=200`: ordered chunks and provenance.
- `POST /api/vector/search`: `query`, optional `top_k` (1–100), `category`, `doc_id`. Returns scores, text and available source-page/clause/line metadata.

The search implementation reads a FAISS inner-product index, embeds the query, retrieves candidates, filters by document/category and excludes deleted documents. A dimension mismatch returns HTTP 409 and triggers a rebuild task. An absent index or unavailable embedding model must be resolved before successful semantic retrieval.

Index compatibility and result: The saved 1,024-dimensional Voyage 4 index accepts local `voyageai/voyage-4-nano` query embeddings with `EMBEDDING_QUERY_MODEL=voyageai/voyage-4-nano`; no index rebuild is needed. Set `EMBEDDING_DOCUMENT_MODEL=voyageai/voyage-4-nano` to encode newly added documents locally. Download Nano once with `LOCAL_EMBEDDING_LOCAL_ONLY=0`, then set `1` for offline use. Search returned HTTP 200 with three hits; an isolated append check grew a copy of the index from 5,719 to 5,720 vectors.

Maintenance endpoints are backend operations: `POST /api/kb/build` (combined), `/api/vector/rebuild` (vector only), `/api/graph/rebuild` (graph only). Poll `GET /api/kb/build/<task_id>` for stages, logs and result. A returned task ID means accepted, not completed. Existing indexes were not deliberately rebuilt for this manual.

8. Knowledge graph

Open **知识图谱**. Enter equipment, a metric, a fault or standard in **统一检索**, then click **搜索** or press Enter. The page requests a subgraph and evidence chains. Suggestions and topic chips focus matching entities. Select a node to inspect/focus it; drag/zoom the D3 network and use **+**, **−**, **适配**. **刷新图谱** reloads; **图谱大屏展示/回到总览** reset filters.

The visible workspace includes node/edge counts, typed-node counts, current zoom, search results and evidence-chain summaries. Backend entity lookup supports name/type and query limits; chain lookup supports seed limit, chain limit and one/two hops. Typed extraction includes `StandardClause`, `FaultMode`, `InspectionTask`, `MaintenanceAction`, `ThresholdRule`, with relationships linking evidence, faults, inspections and actions.

Graph setup and result: Set `NEO4J_URI`, `NEO4J_USER`, and `NEO4J_PASSWORD` for a running Neo4j instance, then restart Flask. The connected graph contains 380 nodes and 1,314 relationships. `/api/graph/overview` returned HTTP 200; the interface displayed its global metrics, a focused network, entity details and evidence chains. Searching 变压器 returned ten chains. If constructing a graph from saved vector chunks, use `KG_EXTRACT_MODE=rule` and `POST /api/graph/rebuild` against a backed-up or isolated database, and follow progress at `GET /api/kb/build/<task_id>`.



Populated graph and selected entity



Graph search with evidence chains



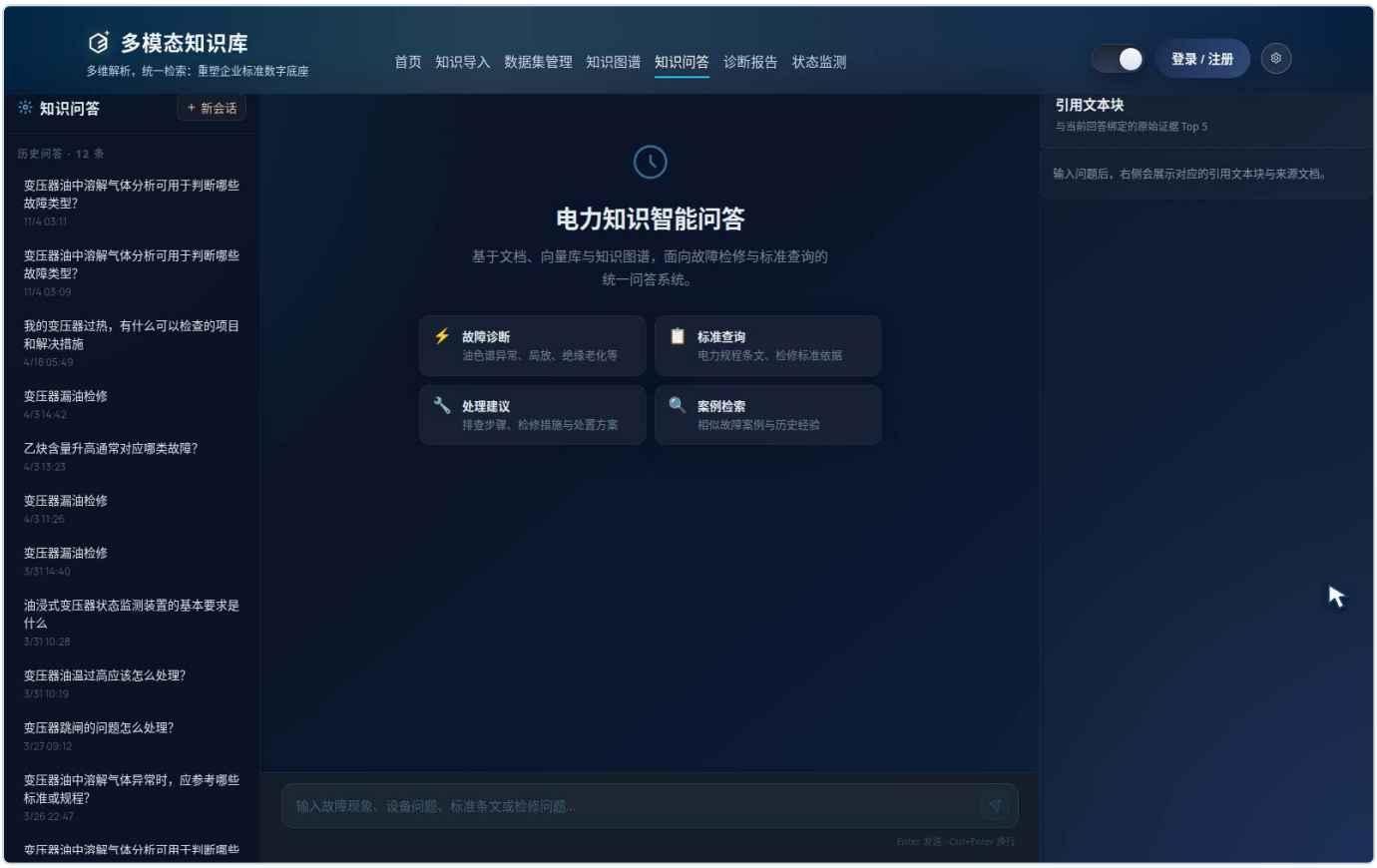
Evidence-chain results and entity details

9. Evidence-based questions

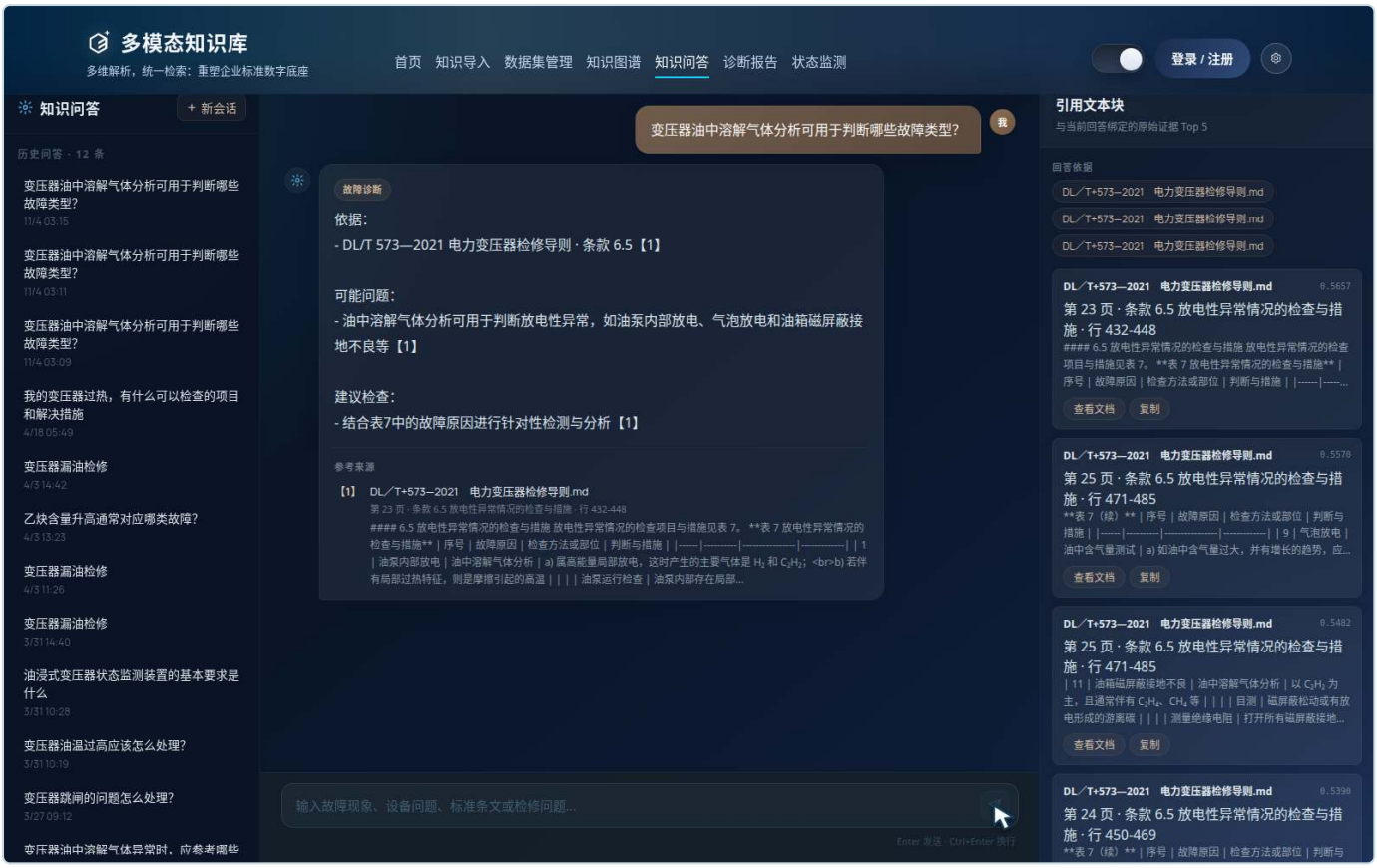
Open **知识问答**. Choose a suggested category/question or type a question. **Enter** sends; **Ctrl+Enter** inserts a line break. The send button is disabled during execution. **新建对话** starts a fresh conversation; selecting history reopens a saved answer. The history context menu supports deletion of a saved question.

Answers can include inline 【1】 citations, source-document links and a right-hand **引用文本块** panel with up to five evidence excerpts. **查看文档** opens the associated dataset source; **复制** copies the excerpt. Inspect page, clause, section and line metadata where present. These fields describe available source provenance, not a guarantee that each old record has a resolvable PDF.

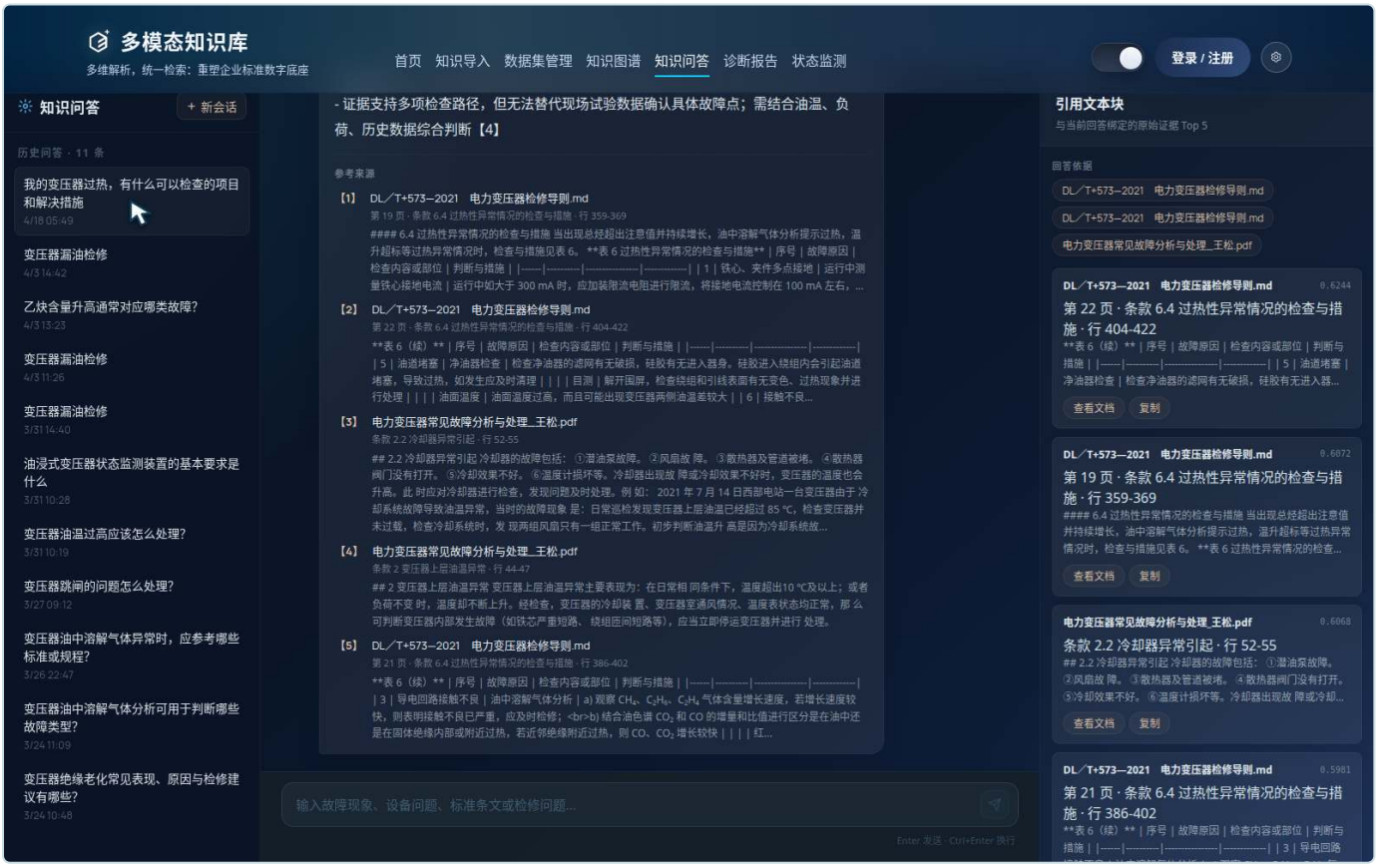
Configuration and result: With `EMBEDDING_QUERY_MODEL=voyageai/voyage-4-nano`, the saved Voyage 4 index accepts a local 1,024-dimensional query vector. Configure a working answer model such as local Ollama `qwen2.5:7b`. The captured DGA question returned an answer with inline citations, document links and a populated evidence panel. The other answer screenshots show saved history.



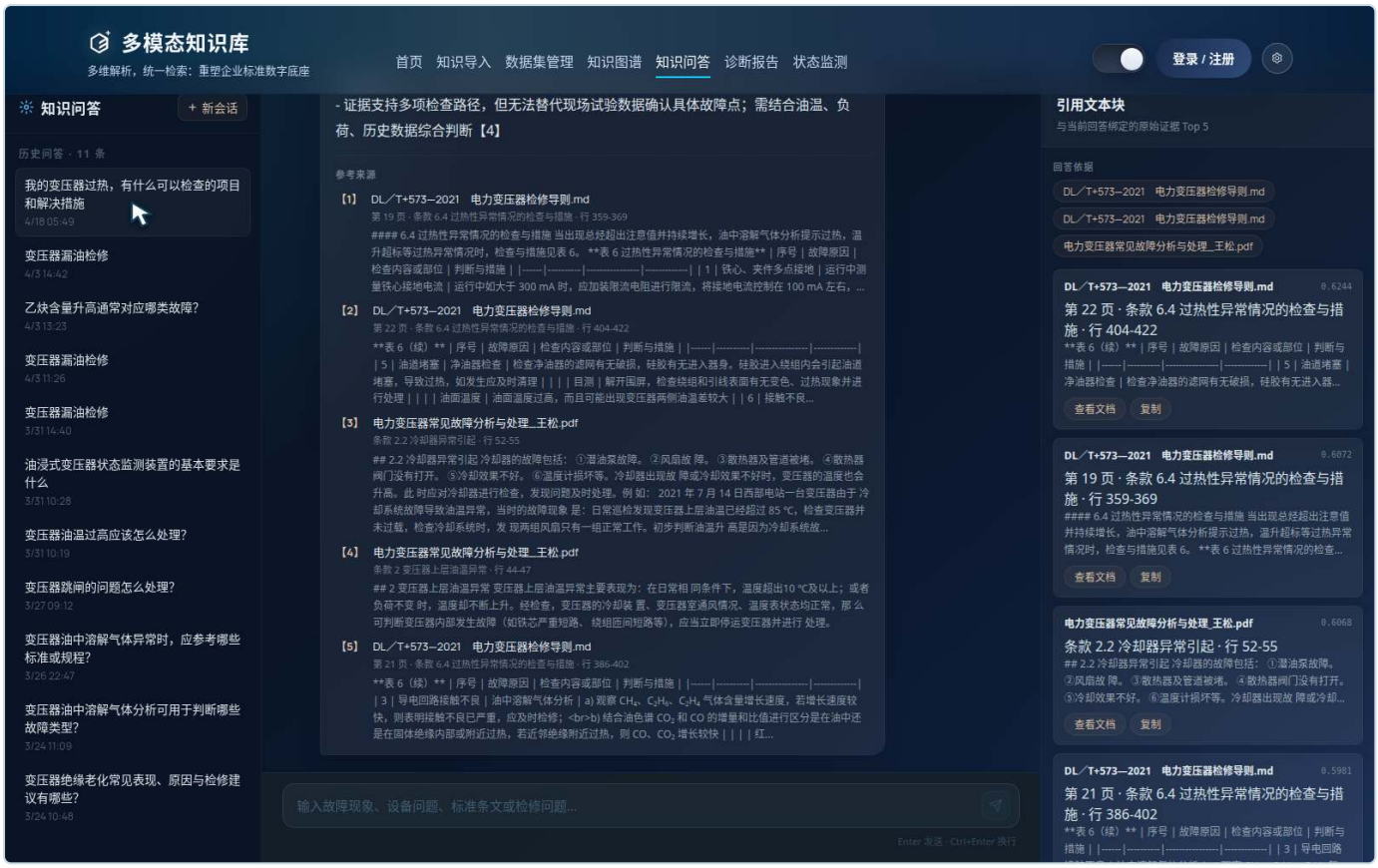
Question workspace and history



Fresh answer with inline citations



Previously saved answer with inline citations



Historical evidence excerpts

10. Diagnosis: three input modes

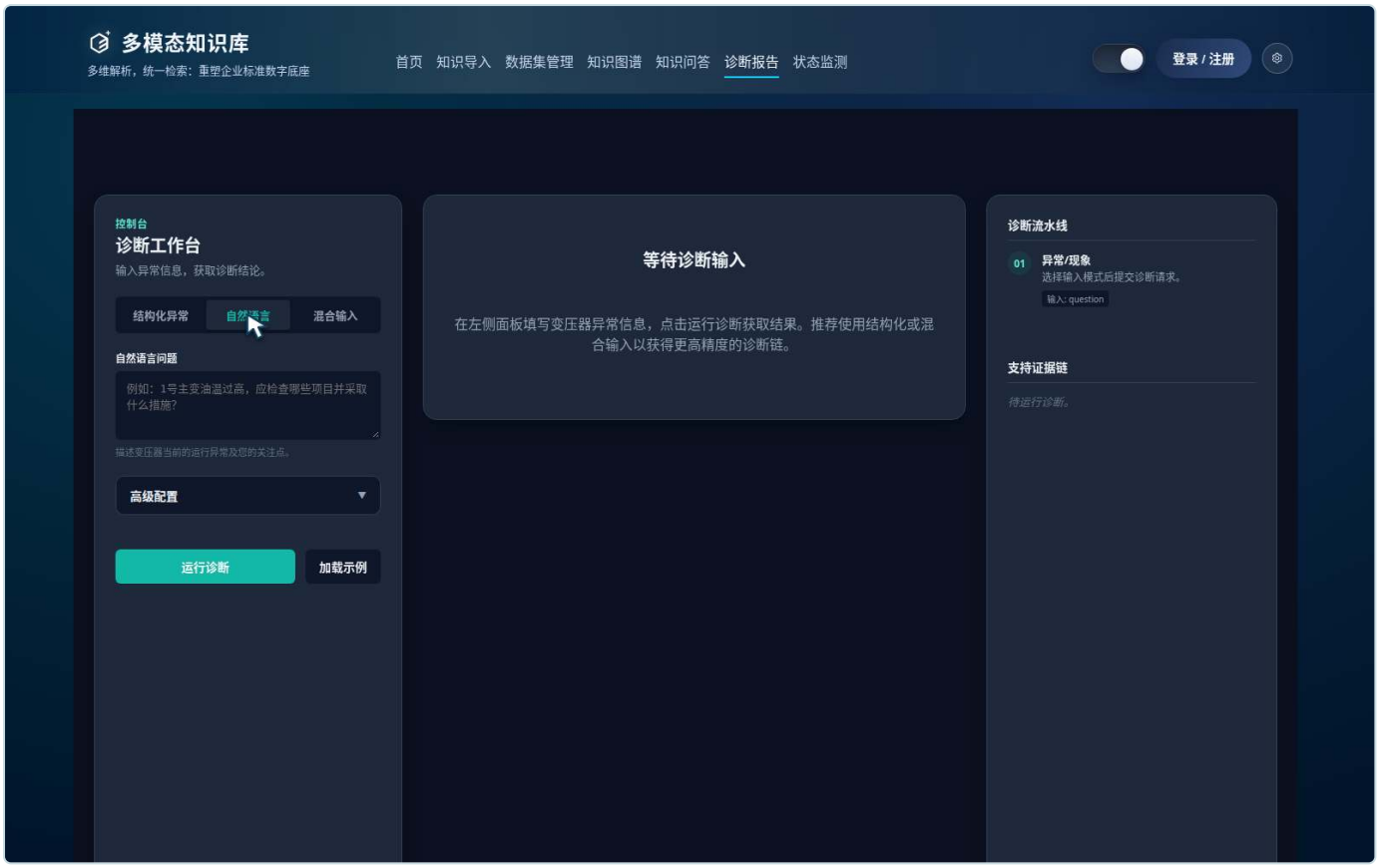
Open **诊断报告** (`/diagnosis`). The tabs select **结构化异常**, **自然语言**, or **混合输入**. Natural language supplies a question; structured mode supplies equipment, symptoms/signals and measured values; hybrid combines both. Switching modes changes the form used for submission.

Structured fields include equipment ID/name/type, station, component, model and voltage; observed phenomena and one-item-per-line symptoms/signals; measurement ID/name/key, value, unit, timestamp and source. Add measurements with **+ 添加测量**, or **移除** an extra row. Enter attention/warning/alarm thresholds and select **数值越高越危险** or **数值越低越危险**. History accepts one `timestamp,value` pair per line. Advanced configuration exposes case ID, retrieval `top_k` and debug output.

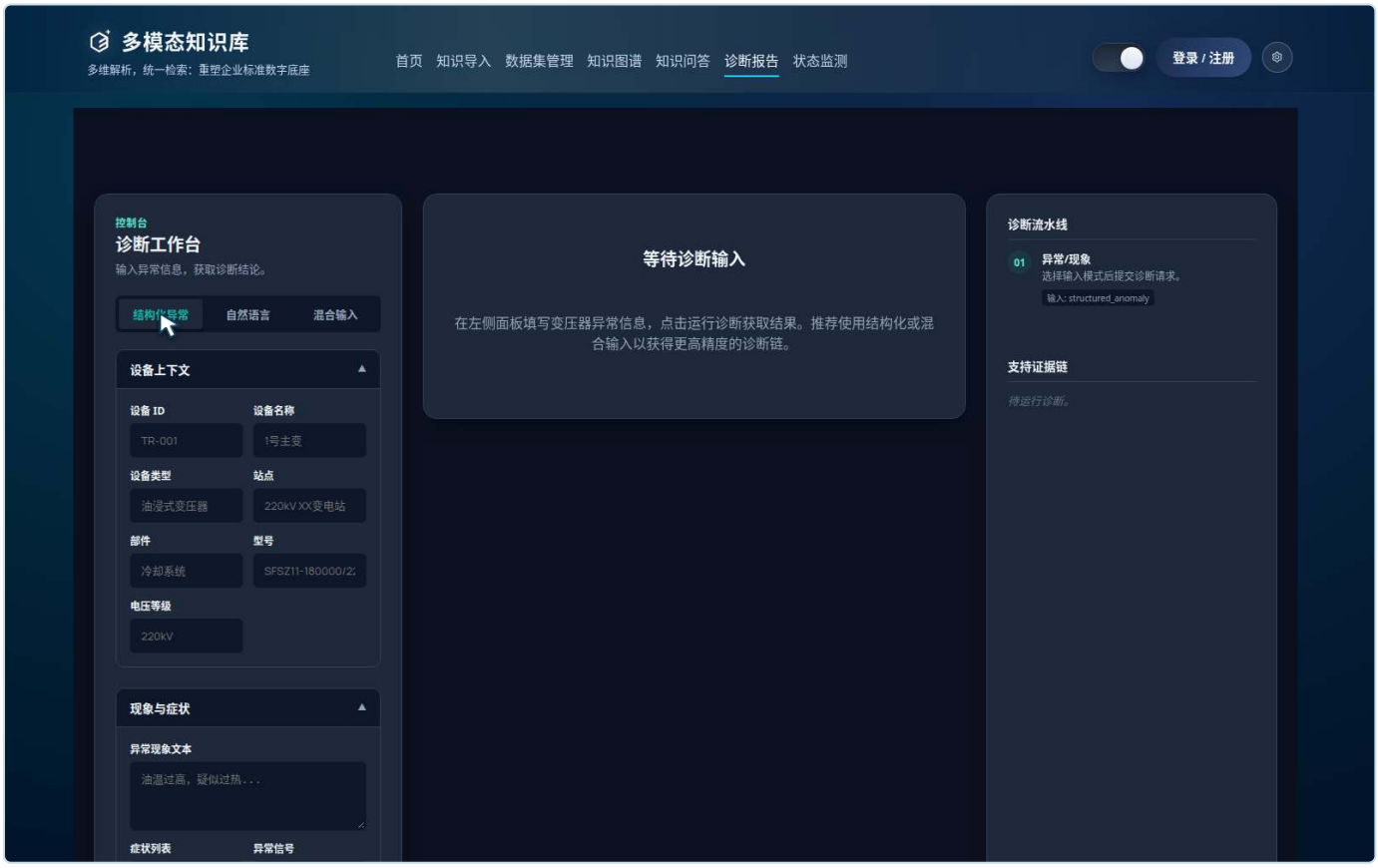
Reproduce the completed demo: click **加载示例**, inspect its two measurements, then **运行诊断**. The built-in example uses hybrid input. It supplies top-oil temperature 95°C (thresholds 85/90/95) and load rate 87% (80/85/95). The actual output classified them as **alarm** and **warning**, respectively, and overall risk as **Critical**. A persistent report was generated.

Read the result's anomaly summary, fault candidates, checks, maintenance actions, applicability, contradictions, missing evidence and rule hits. Upper/lower threshold direction changes the comparison; a missing threshold is not an automatic normal finding. Evidence-enriched thresholds and discrepancies are displayed when available.

Interpretation of this run: The synthetic example returned six vector evidence chunks (`evidence_count=6`). Its oil-temperature topic did not match a typed graph candidate in the connected graph (`candidate_count=0`), so the diagnosis used the input measurements, retrieved text and rule guidance. The saved report contains two measurement facts, 12 standard-clause excerpts and one vector threshold reference. The 85/90/95 thresholds were example inputs, not newly retrieved standard limits; fault candidates are suggestions rather than confirmed diagnoses.



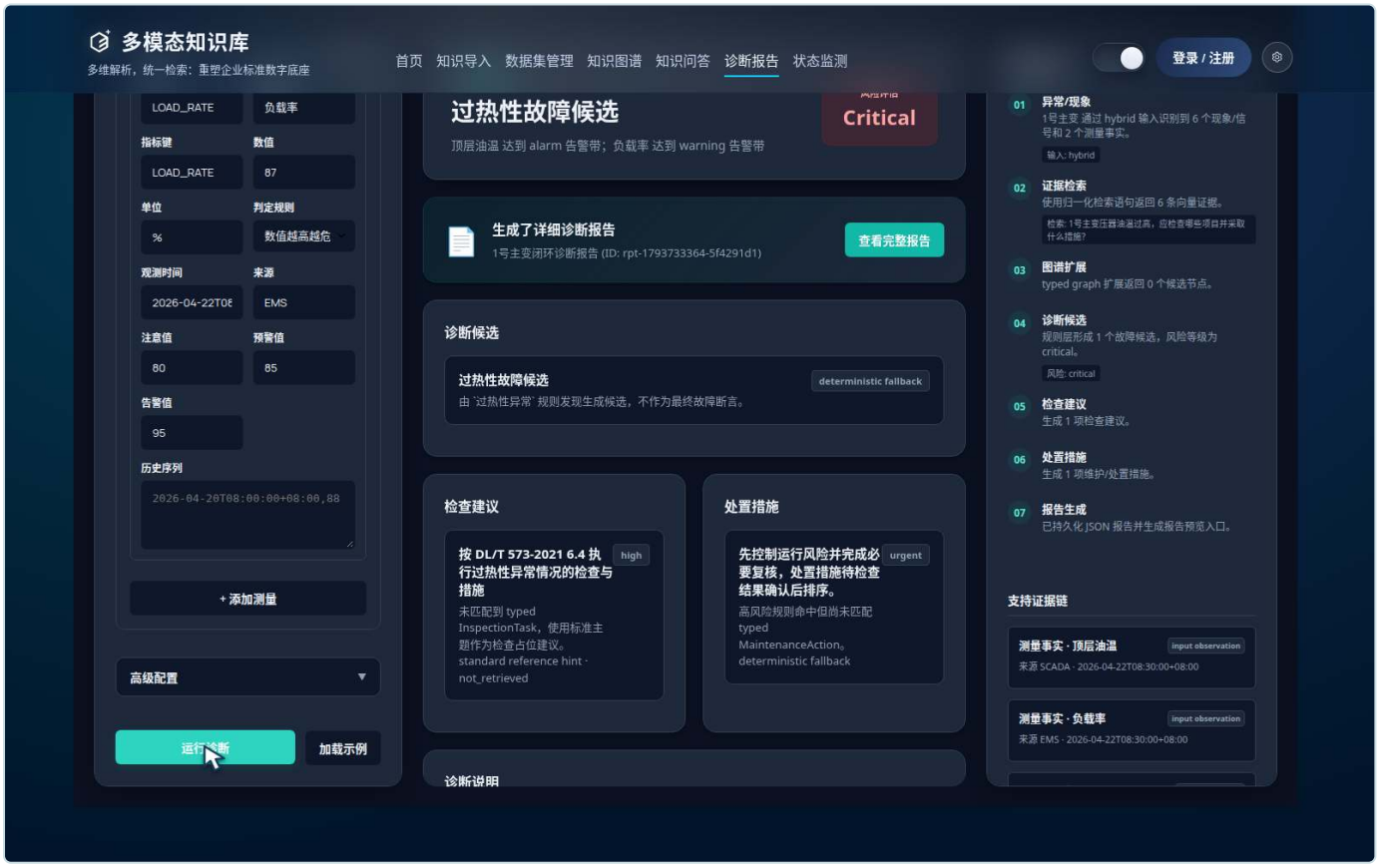
Natural-language input



Structured anomaly form



Loaded hybrid example



Actual completed diagnosis and report link

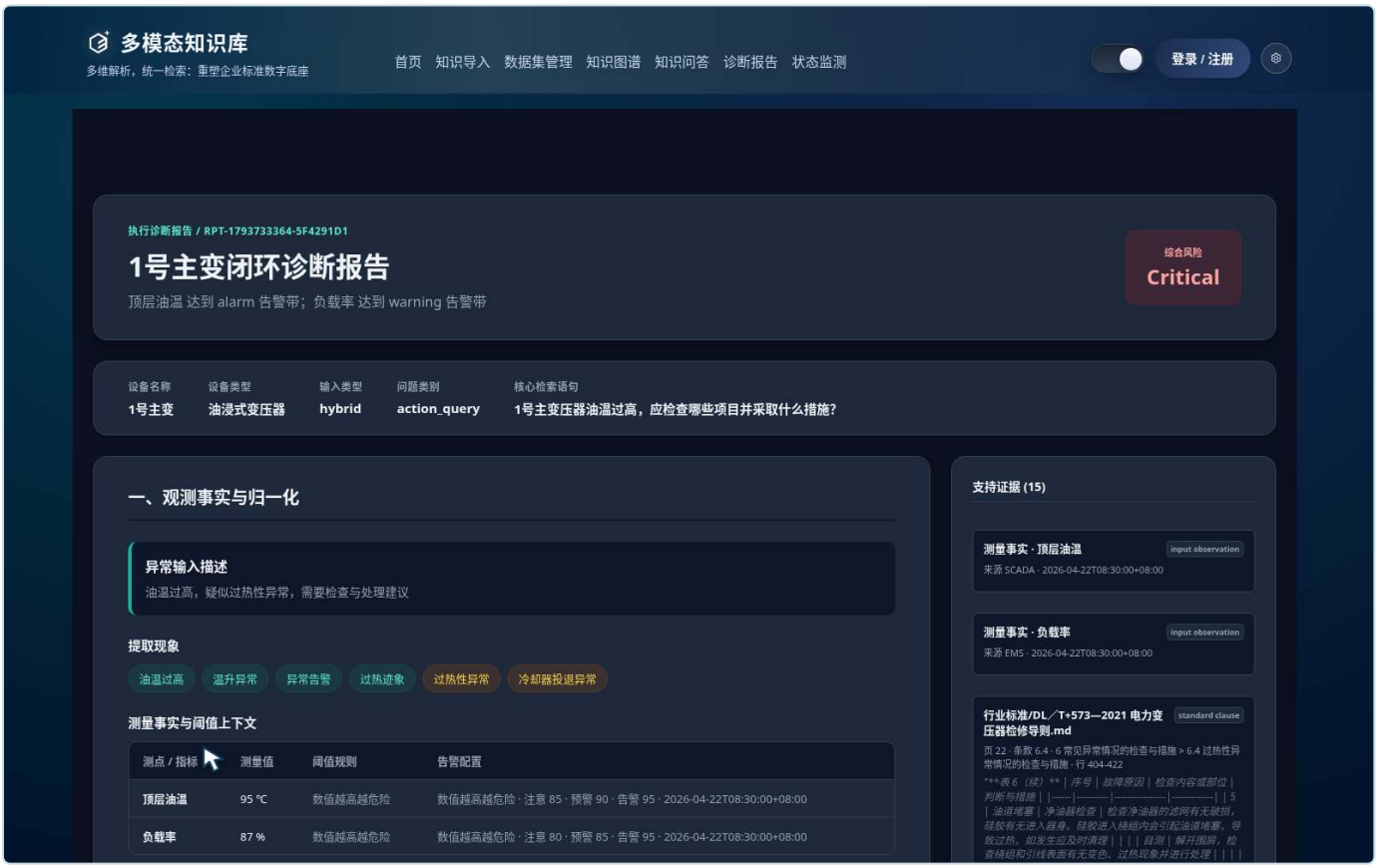
11. Saved reports and source evidence

Click **查看完整报告** after a successful run. The saved-report route uses the report ID returned by that run. The report presents input facts and normalized measurements, thresholds/rules, candidates, inspections/actions, and supporting evidence/technical details. Expand **四、附加追踪与技术细节** to inspect pipeline and enrichment notes. Scroll through the full report; the screenshots show its beginning and lower sections.

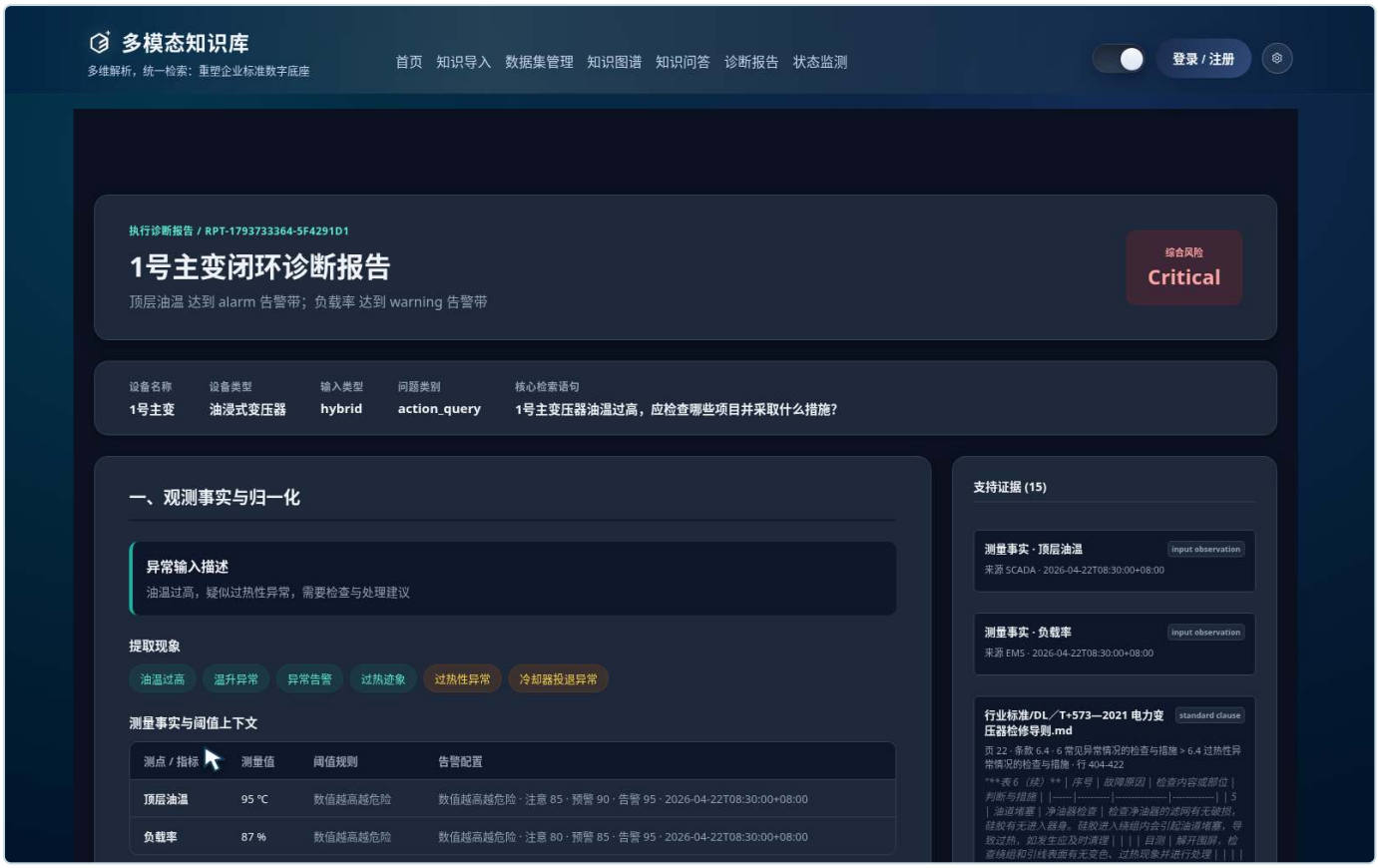
Reports are persisted as local JSON under `uploads/reports/`. List them with `GET /api/reports`, load one with `GET /api/reports/<report_id>`, or rebuild a report from an available run using `POST /api/reports/from-run/<run_id>`. Run detail is held in process memory, while saved report files survive restart. Keep the report ID to reopen the document.

Evidence buttons open a resolved PDF reference or the dataset document when source metadata is available. `/api/references/pdf/<token>` requires a registered token. This report lists two measurement facts, 12 standard-clause excerpts and one vector threshold reference.

The application has no dedicated report PDF-export button in this view. The Chinese/English PDFs accompanying this manual are generated documentation. Use browser printing separately if a diagnosis report must be printed.



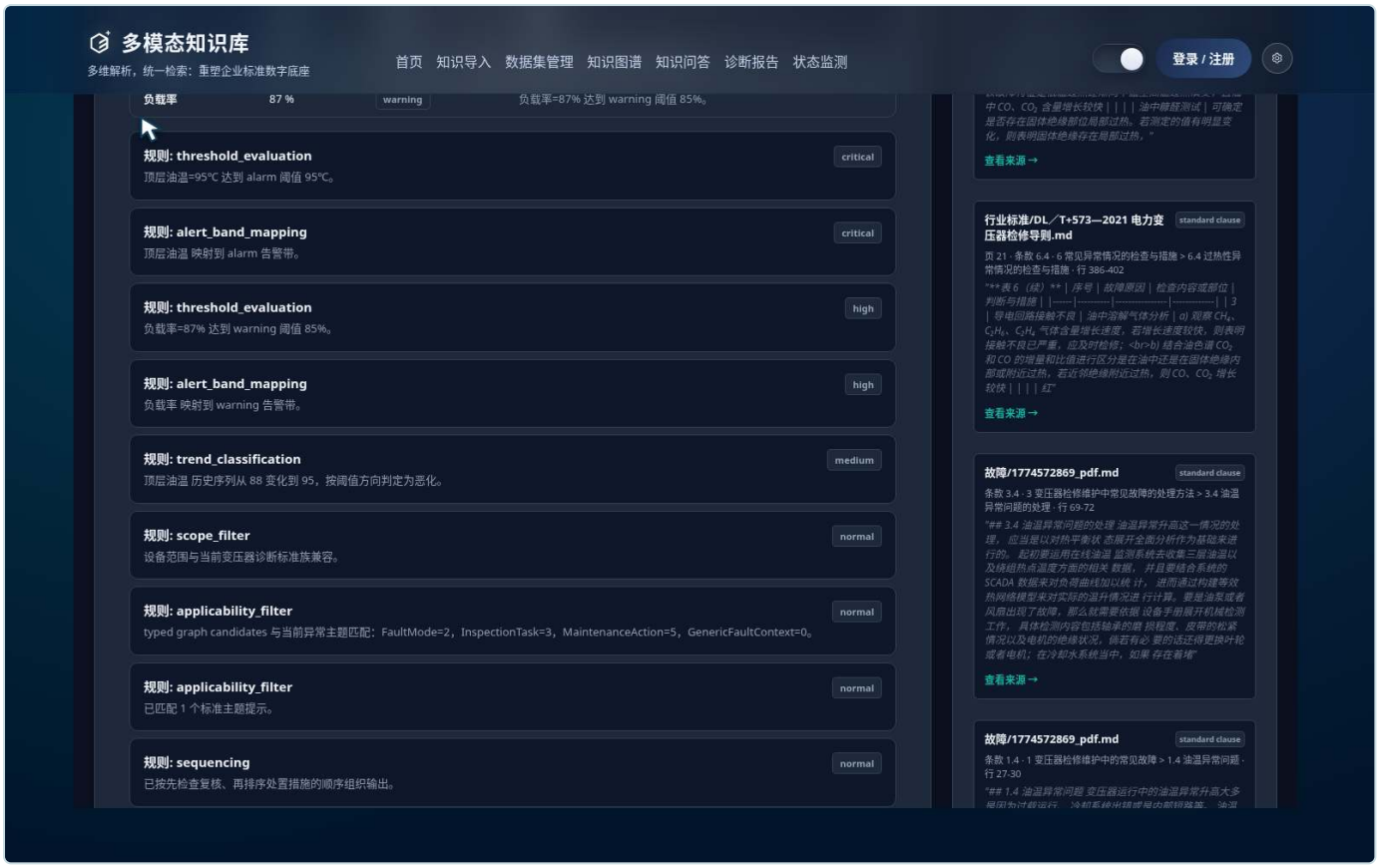
Saved report header and measured facts



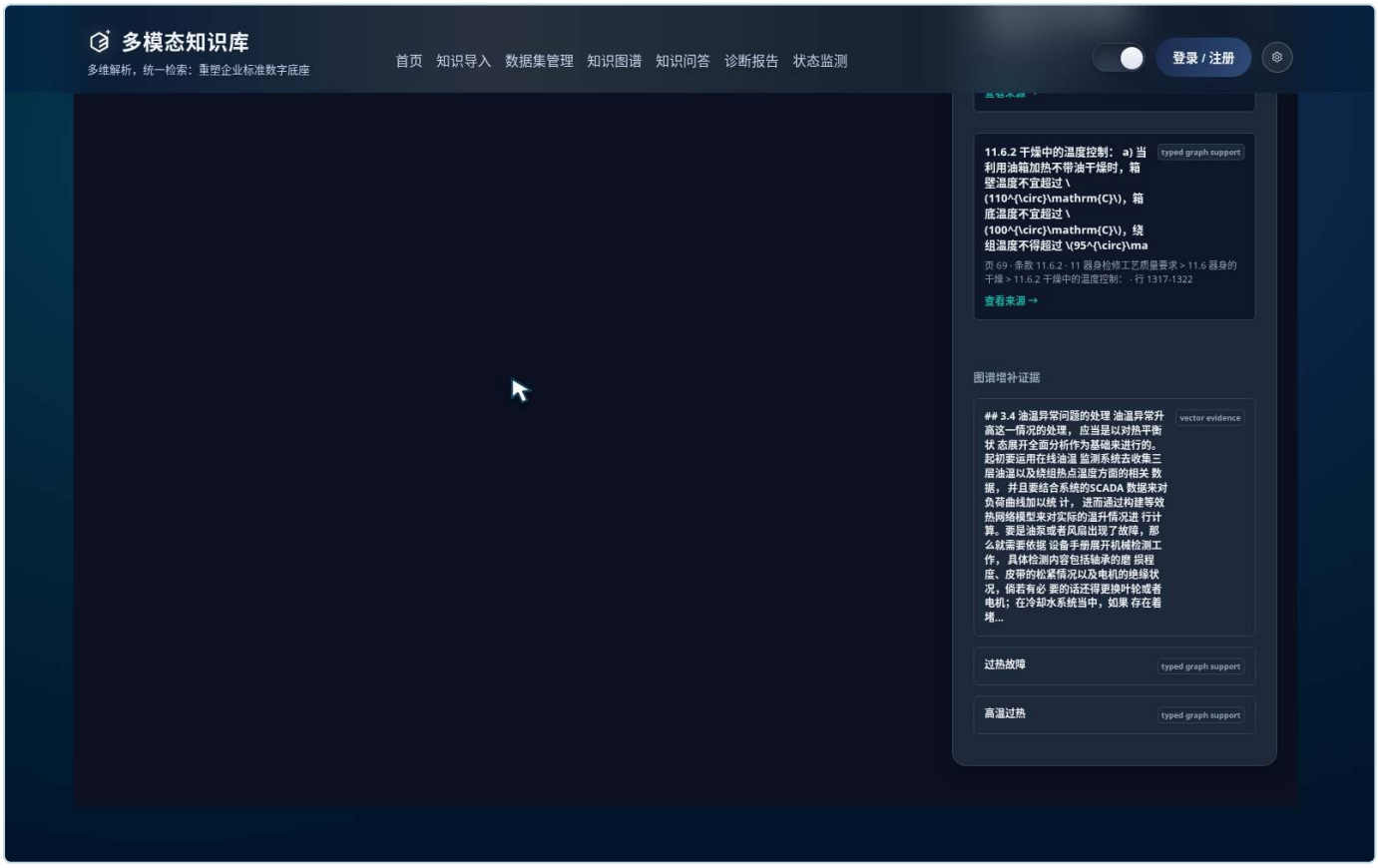
Rule-based reasoning and recommendations



Supporting evidence panel



Fault candidates and action recommendations



Expanded technical details and pipeline stages

12. Condition monitoring

Open **状态监测**. Its separate file picker imports equipment series directly into InfluxDB. Choose an Excel workbook and click **上传并导入时序数据**. Recognized sheet-name markers are **主变压器时序**, **GIS时序**, **断路器时序**, **主变压器运行日志**. Each usable sheet needs `timestamp` or `log_date`, plus `device_id`. The monitoring-demo.xlsx workbook contains 24 synthetic readings for two devices, including `top_oil_temp_c` and `load_mw`.

For **单台设备多指标**, select data type and device, toggle metric chips and choose **归一化对比/原值对比**. Normalization supports shape comparison; read the result table for original values/units. For **单指标多设备**, choose a metric and toggle device chips. **刷新** reruns the current selection; the lower query table shows returned data.

Configuration and result: Start InfluxDB OSS v2 and set `INFLUXDB_URL`, `INFLUXDB_ORG`, `INFLUXDB_BUCKET` and `INFLUXDB_TOKEN` in `.env`, then restart Flask. The isolated demo uses port `8087`; `/api/timeseries/ping` and `/api/timeseries/overview` returned HTTP 200. Uploading `monitoring-demo.xlsx` wrote 24 points for two devices to its demo bucket, and both chart modes displayed populated series. CSV files are supported when their filename contains a recognized measurement marker; `.xls` uses `xlrd`.

On repeat import, the importer replaces points only for each incoming measurement and device within its date interval. The isolated demo bucket is separate from any production time-series database.



Condition monitoring controls



Complete multi-metric chart and data table



Single-metric multi-device controls

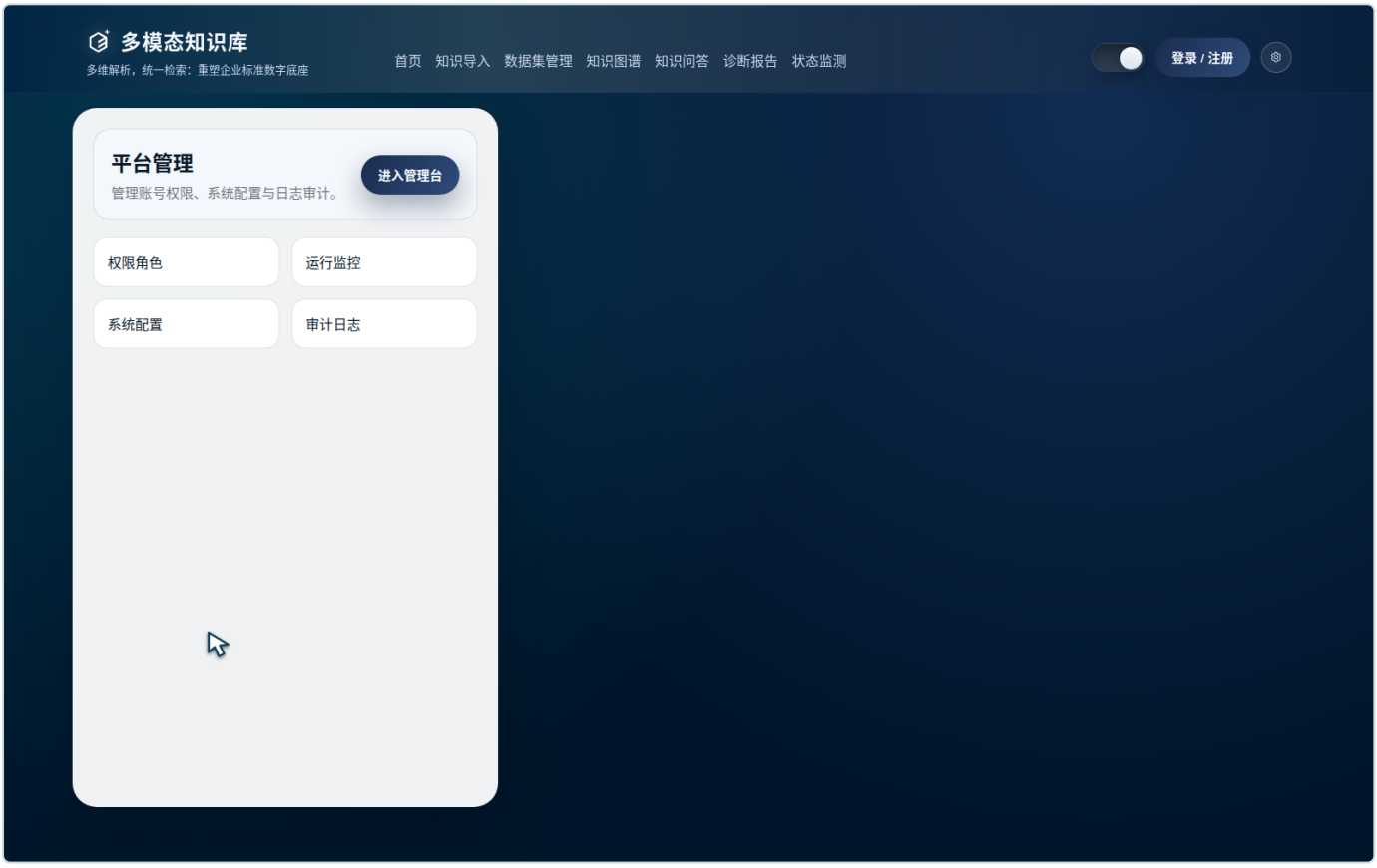


Complete multi-device comparison chart

13. Accounts and management

The top **登录 / 注册** button has no bound click handler in `App.vue`. **设置 → 用户管理** navigates to `/manage`. That page displays permissions, monitoring, configuration and audit cards; **进入管理台** has no bound action. **退出** returns home without calling the authentication logout endpoint. These visible controls are not a completed account-management workflow.

The backend does implement combined email-code registration/login: **send code → verify code → store issued session tokens → refresh → logout**. Its routes are `/api/auth/send-code`, `/verify-code`, `/refresh`, `/logout`. Verification creates the user when necessary. SMTP, authentication settings and the local SQLite store support this service. See auth_api_quickstart.md for request/response fields. No email was sent and no real account was created for this manual.

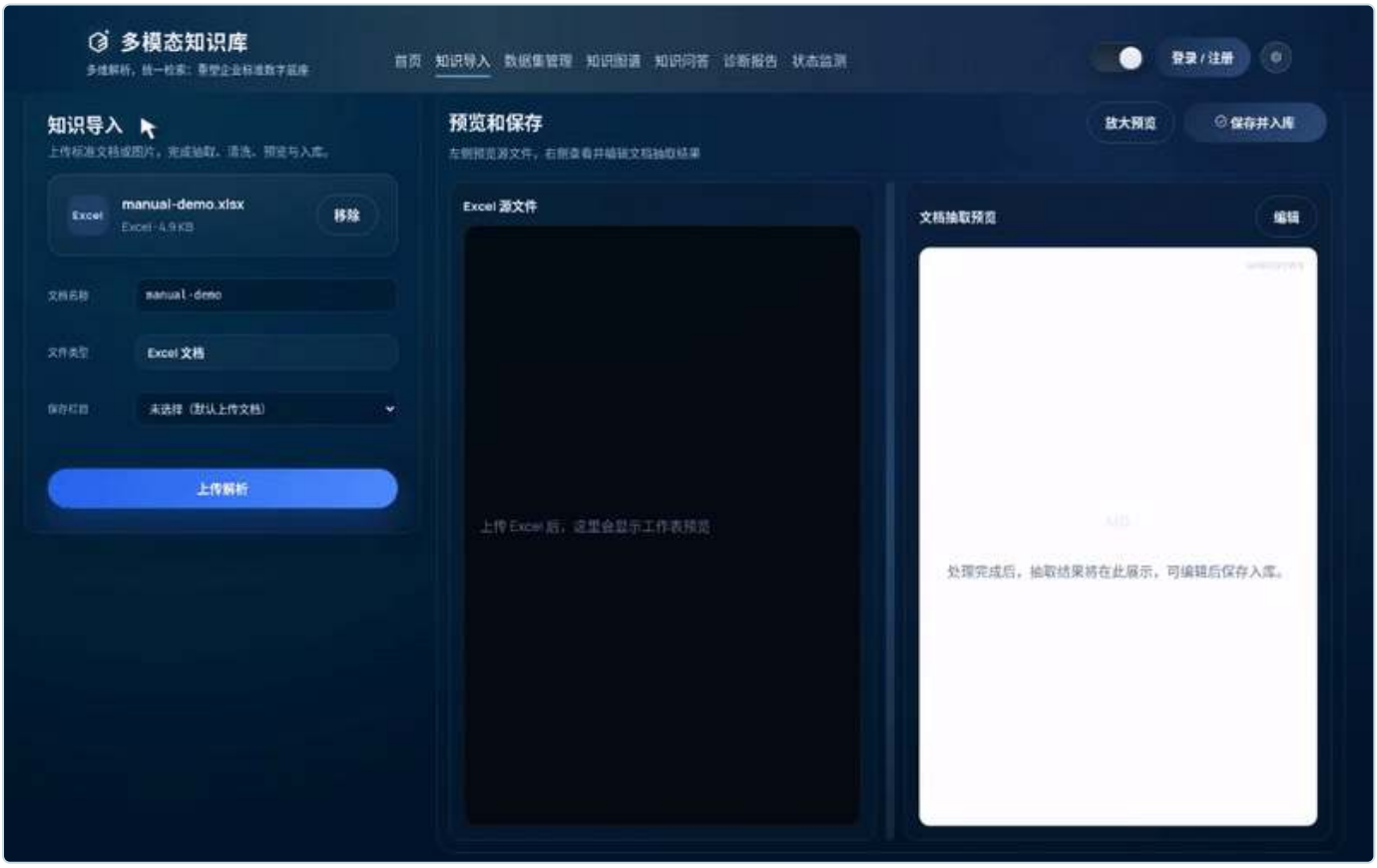


Current management entry page

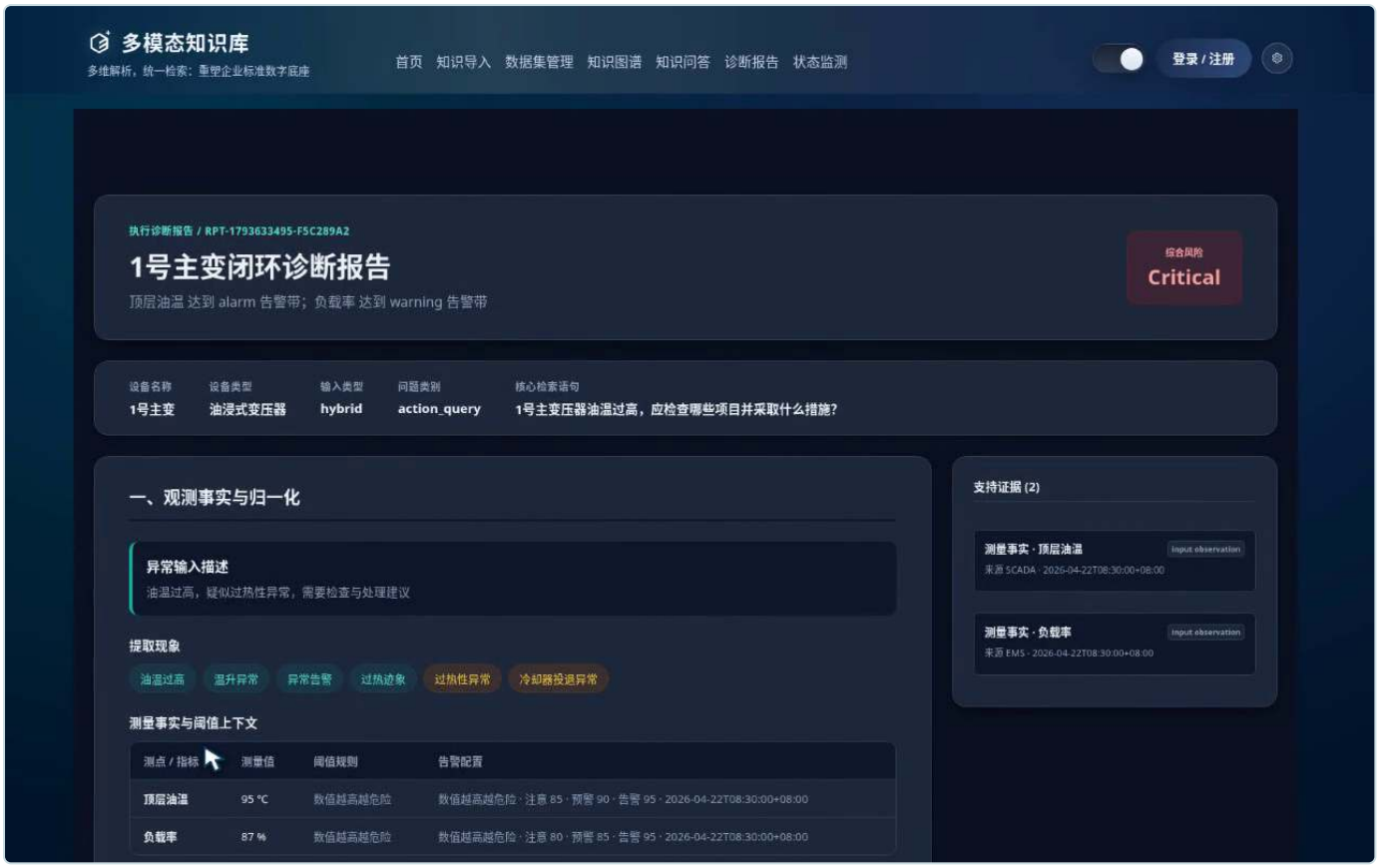
14. Recordings and screen grabs

The following are actual, silent browser recordings. They move the cursor smoothly along a timed path before each interaction, mark clicks with a pulse, and play at normal speed. Actual waiting states remain visible. MP4 links are broadly compatible; Markdown video embedding depends on the viewer. PDFs show still images and retain video links, not playable embedded video. Keep the `videos/` folder beside the guides.

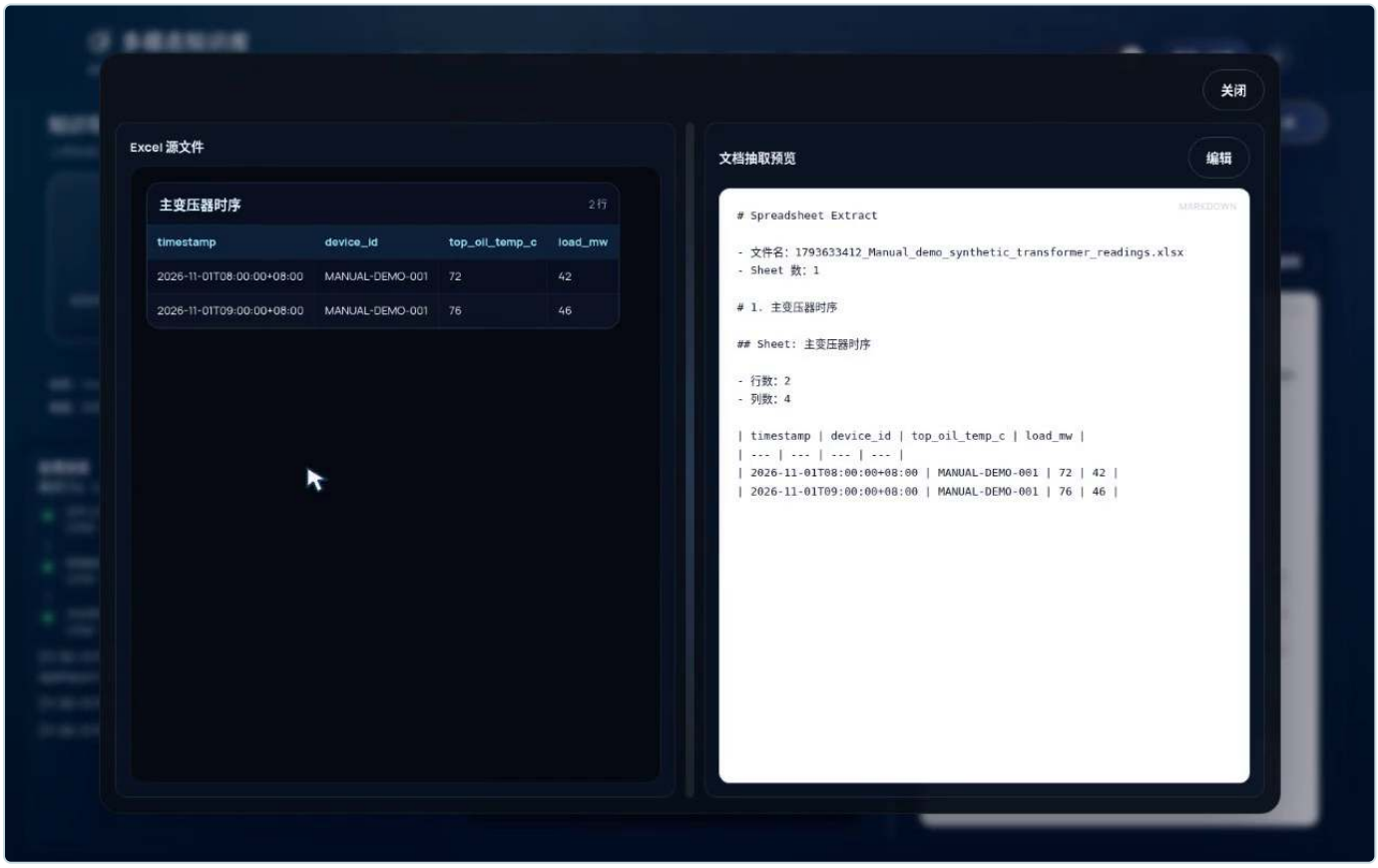
Recording	Content
Interface tour — MP4	Home/theme/settings; import; dataset PDF/extraction/chunks; graph search; fresh Q&A answer; completed diagnosis/report; populated monitoring and management
Import and library — MP4	Real spreadsheet upload, extraction, draft editing, commit and image preview
Saved evidence — MP4	Historical answer/citations, diagnosis modes, report evidence, workbook source and edited extraction
Report details — MP4	Scroll through reasoning/actions and expand technical pipeline stages
Library actions — MP4	Search, rename, soft delete, API-assisted restore, source preview, category filtering, pagination and sidebar collapse
Monitoring charts — MP4	Scroll through populated multi-metric and multi-device charts



Animated import screen recording



Frame grabbed from the actual diagnosis recording



Frame grabbed from the actual import recording



Frame grabbed from the populated graph recording



Frame grabbed from the monitoring recording

Original WebM recordings are also retained for reproducibility. The recordings demonstrate screen workflows; they are distinct from the system's video-file parsing feature.

15. API examples and endpoint catalogue

Use the backend port for direct API calls. JSON operations return structured success/error fields; inspect both HTTP status and `ok`. Uploads use multipart form data. Replace IDs/paths with actual returned values.

```
curl http://127.0.0.1:5002/api/vector/overview
curl http://127.0.0.1:5002/api/reports
curl -X POST http://127.0.0.1:5002/api/vector/search \
  -H 'Content-Type: application/json' \
  -d '{"query":"变压器油温过高","top_k":5}'
```

The submitted hybrid request is retained as diagnosis-request.json; the actual output is diagnosis-run.json.

```
curl -X POST http://127.0.0.1:5002/api/diagnosis/run \
  -H 'Content-Type: application/json' \
  --data-binary @docs/manual/samples/diagnosis-request.json
```

That POST creates a new local report. The schema/example/validate endpoints concern the diagnostic schema contract; `/api/diagnosis/run` accepts the runtime envelope with `input_type`, `question`, `structured_input`, `top_k`, `case_id`, `include_debug`. Do not submit the schema example directly as a runtime request without checking its structure.

The following catalogue is generated from all Flask route decorators in `app.py` and the registered vector/time-series/auth modules. **Listed** means implemented in source, not that every mutation or service-dependent route succeeded. The source location identifies its actual handler. The broad API fallback is also included.

Method	Route	Handler / source
GET	/	index · app.py:1906
GET	/api/pdfs	list_pdfs · app.py:1914
POST	/api/pdfs/reindex	reindex_pdfs · app.py:1974
POST	/api/pdfs/rename	rename_pdf · app.py:1989
POST	/api/pdfs/delete	soft_delete_pdf · app.py:2020
GET	/api/pdfs/deleted	list_deleted_pdfs · app.py:2087
POST	/api/pdfs/restore	restore_deleted_pdf · app.py:2108
GET	/api/pdfs/markdown	pdf_markdown_preview · app.py:2170
GET	/api/fields	list_fields · app.py:2207
POST	/api/upload-document	upload_document · app.py:2212
GET	/api/upload-document/<task_id>	upload_document_status · app.py:2283
POST	/api/upload-document/<task_id>/reparse	upload_document_reparse · app.py:2322
GET	/api/upload-document/<task_id>/source	upload_document_source · app.py:2390
GET	/api/upload-document/<task_id>/spreadsheet-preview	upload_document_spreadsheet_preview · app.py:2459

Method	Route	Handler / source
GET	/api/pdfs/spreadsheet-preview	pdf_spreadsheet_preview · app.py:2509
GET	/api/pdfs/chunks	pdf_chunks_preview · app.py:2532
POST	/api/upload-document/<task_id>/metadata	upload_document_metadata · app.py:2569
GET	/api/upload-history	upload_history · app.py:2584
DELETE	/api/upload-history/<task_id>	upload_history_delete · app.py:2621
GET	/api/upload-markdown/<task_id>	upload_markdown_preview · app.py:2631
POST	/api/upload-markdown/<task_id>/edit	upload_markdown_edit · app.py:2713
POST	/api/upload-document/<task_id>/commit	commit_uploaded_pdf · app.py:2774
POST	/api/kb/build	kb_build · app.py:2805
POST	/api/vector/rebuild	vector_rebuild · app.py:2820
POST	/api/graph/rebuild	graph_rebuild · app.py:2837
GET	/api/kb/build/<task_id>	kb_build_status · app.py:2854
GET	/api/datasets/summary	dataset_summary · app.py:2968
GET	/api/datasets/distribution	dataset_distribution · app.py:3021
GET	/api/datasets/quality	dataset_quality · app.py:3088
GET	/api/datasets/recent-activity	dataset_recent_activity · app.py:3117
GET	/api/graph/overview	graph_overview · app.py:3126
GET	/api/graph/subgraph	graph_subgraph · app.py:3373
GET	/api/graph/entities	graph_entities · app.py:3714
GET	/api/graph/entity	graph_entity_detail · app.py:3786
GET	/api/graph/chain	graph_chain · app.py:3902
POST	/api/qa/ask	qa_ask · app.py:4402
GET	/api/qa/history	qa_history · app.py:4464
DELETE	/api/qa/history/<record_id>	qa_history_delete · app.py:4471
GET	/api/diagnostic/schema	diagnostic_schema · app.py:4478
GET	/api/diagnostic/example	diagnostic_example · app.py:4487
POST	/api/diagnostic/validate	diagnostic_validate · app.py:4514
POST	/api/diagnosis/run	diagnosis_run · app.py:4536
GET	/api/diagnosis/runs/<run_id>	diagnosis_run_detail · app.py:4655
GET	/api/reports	report_list · app.py:4664
GET	/api/reports/<report_id>	report_detail · app.py:4674
POST	/api/reports/from-run/<run_id>	report_from_run · app.py:4682

Method	Route	Handler / source
GET	/api/references/pdf/<token>	reference_pdf · app.py:4720
GET	/api/<path:_path>	api_not_found · app.py:4733
GET	/pdfs/<path:pdf_path>	serve_pdf · app.py:4769
GET	/<path:path>	static_proxy · app.py:4806
GET	/api/vector/docs	vector_docs · core/vector/routes.py:81
GET	/api/vector/overview	vector_overview · core/vector/routes.py:109
POST	/api/vector/search	vector_search · core/vector/routes.py:124
GET	/api/vector/doc-chunks	vector_doc_chunks · core/vector/routes.py:237
GET	/api/timeseries/ping	timeseries_ping · core/timeseries/routes.py:17
GET	/api/timeseries/overview	timeseries_overview · core/timeseries/routes.py:21
GET	/api/timeseries/devices	timeseries_devices · core/timeseries/routes.py:26
GET	/api/timeseries/query	timeseries_query · core/timeseries/routes.py:31
POST	/api/timeseries/import	timeseries_import · core/timeseries/routes.py:39
POST	/api/timeseries/upload	timeseries_upload · core/timeseries/routes.py:53
POST	/api/auth/send-code	auth_send_code · core/auth/routes.py:27
POST	/api/auth/verify-code	auth_verify_code · core/auth/routes.py:42
POST	/api/auth/refresh	auth_refresh · core/auth/routes.py:58
POST	/api/auth/logout	auth_logout · core/auth/routes.py:74

Backend-only operation recipes

Operation	Inputs and result
Reindex library metadata	POST /api/pdfs/reindex; refreshes local file enumeration/title/category metadata, not the FAISS index.
Upload and poll	POST /api/upload-document with file, file_type, document_name, parse_strategy, target_category; poll /api/upload-document/<task_id>.
Reparse	POST .../<task_id>/reparse with parse_strategy; task must exist and be eligible.
Upload source/preview	GET .../<task_id>/source or /spreadsheet-preview; dataset workbook preview uses /api/pdfs/spreadsheet-preview?file=....
Extraction versions	GET /api/upload-markdown/<task_id>?kind=raw or cleaned or edited; edit through POST .../<task_id>/edit.
Commit	POST /api/upload-document/<task_id>/commit with category, use_edited, upload_meta; returns dataset_file.
Metadata	GET /api/fields; POST /api/upload-document/<task_id>/metadata updates upload metadata.
History	GET /api/upload-history; DELETE /api/upload-history/<task_id> removes its history record, not a substitute for dataset deletion.

Operation	Inputs and result
Dataset previews	`/api/pdfs/markdown?file=...&kind=cleaned`; `/api/pdfs/chunks?file=...`
Rename/delete/restore	POST to `/api/pdfs/rename` (`file`, `title`), `/delete` (`file`), `/restore` (`file`); inspect `/deleted`.
Statistics	`/api/datasets/summary`, `/distribution`, `/quality`, `/recent-activity`.
Graph	`/api/graph/overview`; `/subgraph?q=...`; `/entities?q=...&type=...`; `/entity?norm_name=...&type=...`; `/chain?q=...&max_hops=2`.
Q&A	`POST /api/qa/ask` with `question`, `history`; `GET /api/qa/history?limit=12`; delete one record by ID.
Monitoring	`/api/timeseries/ping`, `/overview`, `/devices`; `/query?measurement=...&device_id=...&days=30&limit=5000`; multipart `/upload` or JSON `/import` with server-local `local_path`.
Auth	send-code (`target`, `channel`); verify-code (`target`, `channel`, `code`); refresh/logout with session-token fields described in the auth quickstart.

All exposed endpoint families are covered above. For internal implementation functions, function-inventory.csv lists Python and Vue/JavaScript declarations by source and line. It is a navigation inventory, not a claim that each helper has been individually executed.

16. Command-line tools and troubleshooting

Run from the repository root using the project interpreter. `PYTHONPATH=.` lets scripts import `core` reliably.

```
venv/bin/python main.py --input-root pdfs --output-root pdfs/markdown --clean
venv/bin/python main.py --clean-only
venv/bin/python main.py --full-page-ocr --overwrite --clean
PYTHONPATH=. venv/bin/python scripts/import_timeseries_excel.py /path/to/data.xlsx
venv/bin/python scripts/import_dga_excel.py /path/to/dga.xlsx --skip-ingest
PYTHONPATH=. venv/bin/python scripts/evaluate_retrieval_strategies.py --help
venv/bin/python scripts/eval_kg_extraction.py
```

Tool	What it does / required input
`main.py`	Bulk PDF conversion and optional cleaning; `--clean-only` cleans existing Markdown; `--overwrite` replaces prior output.
`import_timeseries_excel.py`	Uses the same InfluxDB importer and replacement semantics as monitoring. Requires the recognized sheets and live InfluxDB.
`import_dga_excel.py`	First-sheet headers must start `H2, CH4, C2H6, C2H4, C2H2, 故障类型`; generates a knowledge summary and can ingest it. `--category`, `--basename`, `--skip-ingest` control output.
`evaluate_retrieval_strategies.py`	Compares full-document, sliding-window, paragraph and recursive splitting using embedding or TF-IDF retrieval; supports FAISS/NumPy search and configurable chunk size/overlap/top-k. Consult `--help`.
`eval_kg_extraction.py`	Scores typed graph extraction against `docs/kg_extraction_eval_gold.json`; that gold file is absent in this snapshot, so the command needs its dataset before it can run.
Worker modules	`core/doc/pipeline_worker.py` and `multimodal_worker.py` are subprocess entry points used by the upload service, rather than independent operator workspaces.

These maintenance/evaluation commands were documented from source, not rerun to imply fresh experimental results. Files with a `2` suffix are duplicate source artifacts, not additional registered product features.

Appendix: Running fully local

The unavailability states captured in this manual (Neo4j 503, InfluxDB connection refused, Hugging Face cache error in Q&A) are all resolved by running the backing services locally:

Neo4j. Run a local container and point `NEO4J_URI`/`NEO4J_USER`/`NEO4J_PASSWORD` in `.env` at it:

```
docker run -d --name neo4j -p 7474:7474 -p 7687:7687 neo4j:latest
```

InfluxDB. Run a local InfluxDB 2.x container on port 8086 with the org/bucket/token the backend expects (`core/timeseries/service.py`; defaults org `knowledge-base`, bucket `predictive_maintenance`):

```
docker run -d --name influxdb -p 8086:8086 \
  -e DOCKER_INFLUXDB_INIT_MODE=setup \
  -e DOCKER_INFLUXDB_INIT_USERNAME=admin \
  -e DOCKER_INFLUXDB_INIT_PASSWORD=password123 \
  -e DOCKER_INFLUXDB_INIT_ORG=knowledge-base \
  -e DOCKER_INFLUXDB_INIT_BUCKET=predictive_maintenance \
  -e DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=<token matching INFLUXDB_TOKEN in .env> \
  influxdb:2
```

Verify with `curl http://127.0.0.1:8086/health` and the backend's `GET /api/timeseries/ping`, which should return `"ok": true`.

Embedding model. The local embedding path (`EMBEDDING_BACKEND=local`) loads a sentence-transformers model. Pre-populate the Hugging Face cache once, after which the app runs fully offline (`LOCAL_EMBEDDING_LOCAL_ONLY=1` is already the default):

```
venv/bin/python -c "from sentence_transformers import SentenceTransformer; SentenceTransformer('Qwen/Qwen3-Embedding-0.6B', device='cpu')"
```

If huggingface.co is unreachable, prefix the command with `HF_ENDPOINT=https://hf-mirror.com`. On machines without a working CUDA toolchain, load with `device='cpu'` or `CUDA_VISIBLE_DEVICES=""` to avoid Triton GPU compile errors. The model must match the one used to build the saved FAISS index: the index captured here was built with `voyage-4-large` (1,024 dimensions), so for consistent fully-local retrieval set `EMBEDDING_MODEL` to a local 1,024-dimensional model and rebuild via `POST /api/vector/rebuild`. Dimensions matching alone does not make models interchangeable.

LLM for Q&A and graph extraction. Answer generation and KG relation extraction call a chat LLM. For a fully local setup, run Ollama and configure `.env`:

```
LLM_PROVIDER=ollama
LLM_MODEL=qwen2.5:7b
LLM_BASE_URL=http://127.0.0.1:11434/v1
LLM_API_KEY=ollama
```

Graph rebuilds extract relations per chunk through this LLM; use `KG_MAX_CHUNKS` (e.g. `150`) for a quick smoke-test build before a full rebuild, and `KG_EXTRACT_WORKERS` to control concurrency. Graph rebuilds wipe and rewrite the graph (`clean_rebuild`), so a later full rebuild replaces a capped one without duplication.

Symptom	Check / action
Frontend opens but API requests fail	Verify backend port and Vite proxy; distinguish the current repository from another running copy.
Port 5173 occupied	Vite uses `strictPort`; stop the conflicting server or explicitly choose another port and matching backend.
Graph search has no results	Check the Neo4j URI and credentials, then build graph content from the saved chunks with `/api/graph/rebuild` after backing up the database.
Monitoring connection fails	Start InfluxDB OSS v2; set URL/org/bucket/token in `.env`, restart Flask, verify `/api/timeseries/ping`, then import a recognized Excel sheet.
Hugging Face/cache error in Q&A	The saved index uses Voyage 4. Set `EMBEDDING_QUERY_MODEL=voyageai/voyage-4-nano` and download Nano once; configure a working answer model.
HTTP 409 from vector search	Query/index dimensions differ; inspect the returned rebuild task and wait for successful completion.
Upload processed but not visible in dataset	Finish **保存并入库**; inspect task status and `dataset_file`. Extraction success alone is not commit.
Existing item titled 未知	Inspect source/category and use rename to provide title metadata.
Report does not reopen	Use a persisted report ID; a run ID is different and process-local run records can disappear at restart.
Markdown video does not play	Open the linked MP4 directly or use the HTML edition. PDF uses stills.

Evidence files include the route inventory, GET checks, actual diagnosis result, restoration result and visible text captured per screen. Rebuild instructions are in tools/README.md.